# Robotics Autonomy Report

**Longlong Wang**

**February 2026**

**Public Portfolio Version Statement**

**This document is a public portfolio version of an individual coursework project completed by Longlong Wang. It is provided only to demonstrate the author's technical work.**

# Table of Contents

## Contents

## Table of Figures

## Table of Tables

# 1. Task 1 – Robot Tracking with Non-linear Filter

## 1.1 Algorithmic Initialisation

The robot motion is described using the bicycle kinematic model, where the system state is defined as

$$\mathbf{x} = [x \ y \ \theta]^T$$

Given the nonlinear motion and measurement models, an Extended Kalman Filter (EKF [1]) is employed for state estimation. The EKF is selected due to its suitability for nonlinear systems with Gaussian noise assumptions.

The provided datasets contain odometry inputs and landmark measurements, summarised in Table 1.

| File Name | Data Name | Meaning | Class | Size | Unit |
|---|---|---|---|---|---|
| my_input.mat | t | time vector | double | 1 * 613 | s |
| | v | control input linear velocity | double | 1 * 612 | m/s |
| | om | control input angular velocity | double | 1 * 612 | rad/s |
| my_measurement.mat | l | the coordinate of all landmarks | double | 6 * 2 | m |
| | r | measurement range between landmarks and lidar | double | 612 * 6 | m |
| | b | measurement bearing between landmarks and lidar | double | 612 * 6 | rad |

*Table 1. Default Input Dataset Specification*

Table 2 summarises the primary variable initialisation values. The initial state covariance matrix $P_0$ is selected empirically to represent a moderate level of prior uncertainty. The timestep is set to 1 s to maintain dimensional consistency with the provided odometry sampling interval, although the nominal formulation specifies 0.1 s; a brief consistency analysis comparing different timestep assumptions is presented in Section 1.6.

| Variables | Meaning | Size | Components | Init | unit |
|---|---|---|---|---|---|
| dt | Time step | 1 * 1 | - | 1 | s |
| d | Lidar distance between car center and lidar center | 1 * 1 | - | 0 | m |
| x | State | 3 * 1 | x position | 0.5 | m |

| | | | | | |
|---|---|---|---|---|---|
| | | | y position | 1 | m |
| | | | θ heading | 0 | rad |
| P | Initial State Covariance Matrix | 3 * 3 | Initial x position variance | (0.2)^2 | m^2 |
| | | | Initial y position variance | (0.2)^2 | m^2 |
| | | | Initial heading variance | (deg2rad(2))^2 | rad^2 |
| Q | Process noise Covariance Matrix | 2 * 2 | Linear velocity noise variance | (0.007)^2 | (m/s)^2 |
| | | | Angular velocity noise variance | (0.0075)^2 | (rad/s)^2 |
| R | Measurement Noise Covariance Matrix | 2 * 2 | Range measurement variance | (0.02)^2 | m^2 |
| | | | Bearing measurement variance | (0.15)^2 | rad^2 |

*Table 2. Variable Initialisation Specification*

## 1.2 Prediction Step

### *1.2.1 Motion Model Formulation*

The robot state vector is defined as

$$\mathbf{x}_k = \begin{bmatrix} x_k \\ y_k \\ \theta_k \end{bmatrix}$$

where $x_k, y_k$ denote the global position and $\theta_k$ is the heading angle.

The control input is

$$\mathbf{u}_{k-1} = \begin{bmatrix} v_{k-1} \\ \omega_{k-1} \end{bmatrix}$$

where $v$ and $\omega$ represent the linear and angular velocity obtained from odometry.

Using the discrete-time bicycle model provided in the assignment, the nonlinear state equation can be written as:

$$\mathbf{x}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}, \mathbf{w}_{k-1}) = \mathbf{x}_{k-1} + T \begin{bmatrix} \cos\theta_{k-1} & 0 \\ \sin\theta_{k-1} & 0 \\ 0 & 1 \end{bmatrix} (\mathbf{u}_{k-1} + \mathbf{w}_{k-1})$$

where $T$ is the sampling time and the process noise satisfies

$$\mathbf{w}_{k-1} \sim \mathcal{N}(0, \mathbf{Q}), \mathbf{Q} = \begin{bmatrix} \sigma_v^2 & 0 \\ 0 & \sigma_\omega^2 \end{bmatrix}.$$

### *1.2.2 State Prediction*

In the EKF prediction step, the process noise is assumed zero-mean and is not injected directly into the state estimate. The predicted state is therefore computed as:

$$\hat{\mathbf{x}}_k^- = f(\hat{\mathbf{x}}_{k-1}, \mathbf{u}_{k-1}, 0)$$

which yields the explicit form:

$$\begin{aligned} x_k^- &= x_{k-1} + T v_{k-1} \cos\ \theta_{k-1} \\ y_k^- &= y_{k-1} + T v_{k-1} \sin\ \theta_{k-1} \\ \theta_k^- &= \theta_{k-1} + T\omega_{k-1} \end{aligned}$$

The heading angle is normalised to the interval $(-\pi, \pi]$ using:

$$\theta_k^- = \text{wrapToPi}(\theta_k^-)$$

### *1.2.3 Covariance Prediction*

To propagate the uncertainty, the nonlinear model is linearised around the current estimate. The Jacobian matrices are defined as:

$$\mathbf{F}_{k-1} = \frac{\partial f}{\partial \mathbf{x}} |_{\hat{\mathbf{x}}_{k-1}, \mathbf{u}_{k-1}} = \begin{bmatrix} 1 & 0 & -T v_{k-1} \sin \theta_{k-1} \\ 0 & 1 & T v_{k-1} \cos \theta_{k-1} \\ 0 & 0 & 1 \end{bmatrix}$$

$$\mathbf{\Gamma}_{k-1} = \frac{\partial f}{\partial \mathbf{w}} |_{\hat{\mathbf{x}}_{k-1}, \mathbf{u}_{k-1}} = \begin{bmatrix} T\cos\ \theta_{k-1} & 0 \\ T\sin\ \theta_{k-1} & 0 \\ 0 & T \end{bmatrix}$$

The predicted covariance is then given by:

$$\mathbf{P}_k^- = \mathbf{F}_{k-1} \mathbf{P}_{k-1} \mathbf{F}_{k-1}^T + \mathbf{\Gamma}_{k-1} \mathbf{Q}_{k-1} \mathbf{\Gamma}_{k-1}^T$$

This completes the EKF prediction step, providing both the prior state estimate and its associated uncertainty.

## 1.3 Measurement and Update Step – Single Landmark

### *1.3.1 Measurement Model*

The robot is equipped with a LIDAR sensor that provides range and bearing measurements of known landmarks in the environment.

For landmark $l$, the measurement vector is defined as

$$\mathbf{z}_k^l = \begin{bmatrix} r_k^l \\ \phi_k^l \end{bmatrix}$$

According to the assignment specification (Eq. 1.5), the nonlinear measurement model is:

$$\mathbf{z}_k^l = \begin{bmatrix} \sqrt{(x_l - x_k - d\cos\,\theta_k)^2 + (y_l - y_k - d\sin\,\theta_k)^2} \\ \text{atan2}\,(y_l - y_k - d\sin\,\theta_k, x_l - x_k - d\cos\,\theta_k) - \theta_k \end{bmatrix} + \mathbf{v}_k^l$$

where

$$\mathbf{v}_k^l \sim \mathcal{N}(0, \mathbf{R})$$

and:

- $x_l, y_l$ are the known global coordinates of landmark $l$
- $x_k, y_k, \theta_k$ represent the robot pose
- $d$ is the distance between the robot centre and the LIDAR sensor

Although setting $d = 0$ simplifies the mathematical expression mentioned in assignment, retaining the offset term improves the generality of the model. Therefore, keeping $d$ explicitly in the formulation ensures that the estimation framework remains applicable to more realistic configurations.

*1.3.2 Predicted Measurement*

Using the predicted state $\hat{\mathbf{x}}_k^-$, the predicted measurement is:

$$\hat{\mathbf{z}}_k^{l-} = h(\hat{\mathbf{x}}_k^-)$$

Define:

$$\Delta x = x_l - \hat{x}_k^- - d\cos\hat{\theta}_k^-$$

$$\Delta y = y_l - \hat{y}_k^- - d\sin\,\hat{\theta}_k^-$$

Then:

$$\hat{\mathbf{z}}_k^{l-} = \begin{bmatrix} \sqrt{\Delta x^2 + \Delta y^2} \\ \text{atan2}\,(\Delta y, \Delta x) - \hat{\theta}_k^- \end{bmatrix}$$

The bearing component is wrapped to $(-\pi, \pi]$.

*1.3.3 Innovation*

The innovation (residual) is defined as:

$$\mathbf{y}_k^l = \mathbf{z}_k^l - \hat{\mathbf{z}}_k^{l-}$$

The angular residual is normalised:

$$y_\phi = \text{wrapToPi}(y_\phi)$$

*1.3.4 Measurement Jacobian*

The measurement Jacobian is obtained by linearising $h(\mathbf{x})$ around the predicted state:

$$\mathbf{H}_k^l = \frac{\partial h}{\partial \mathbf{x}} |_{\hat{\mathbf{x}}_k^-}$$

Define:

$$\Delta x = x_l - \hat{x}_k^- - d\cos\hat{\theta}_k^-$$

$$\Delta y = y_l - \hat{y}_k^- - d\sin\hat{\theta}_k^-$$

$$q = \Delta x^2 + \Delta y^2$$

Range component:

$$r = \sqrt{q}$$

$$\frac{\partial r}{\partial x} = -\frac{\Delta x}{\sqrt{q}}$$

$$\frac{\partial r}{\partial y} = -\frac{\Delta y}{\sqrt{q}}$$

$$\frac{\partial r}{\partial \theta} = \frac{d}{\sqrt{q}}(\Delta x \sin\theta - \Delta y \cos\theta)$$

Bearing component:

$$\phi = atan\,2(\Delta y, \Delta x) - \theta$$

$$\frac{\partial \phi}{\partial x} = \frac{\Delta y}{q}$$

$$\frac{\partial \phi}{\partial y} = -\frac{\Delta x}{q}$$

$$\frac{\partial \phi}{\partial \theta} = -1 - \frac{d}{q}(\Delta x \cos\theta + \Delta y \sin\theta)$$

Final Jacobian Matrix:

$$\mathbf{H}_k^l = \begin{bmatrix} -\frac{\Delta x}{\sqrt{q}} & -\frac{\Delta y}{\sqrt{q}} & \frac{d}{\sqrt{q}}(\Delta x \sin\theta - \Delta y \cos\theta) \\ \frac{\Delta y}{q} & -\frac{\Delta x}{q} & -1 - \frac{d}{q}(\Delta x \cos\theta + \Delta y \sin\theta) \end{bmatrix}$$

*1.3.5 EKF Update*

The innovation covariance is

$$\mathbf{S}_k^l = \mathbf{H}_k^l \mathbf{P}_k^- \mathbf{H}_k^{lT} + \mathbf{R}$$

The Kalman gain is

$$\mathbf{K}_k^l = \mathbf{P}_k^- \mathbf{H}_k^{lT} \mathbf{S}_k^{l-1}$$

The posterior state estimate is updated as

$$\hat{\mathbf{x}}_k = \hat{\mathbf{x}}_k^- + \mathbf{K}_k^l \mathbf{y}_k^l$$

and the covariance update is

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k^l \mathbf{H}_k^l)\mathbf{P}_k^-$$

*1.3.6 Implementation*

Both single-landmark and multi-landmark correction steps are implemented within a unified EKF framework to ensure structural consistency. A boolean control variable (single_mode_enabled) is introduced to regulate the number of landmarks incorporated during each correction step.

When enabled, only one landmark measurement is used at each timestep (by default, the first landmark in the dataset), allowing controlled evaluation of single-sensor update behaviour. When disabled, all available landmark measurements are processed sequentially.

This design enables a fair and consistent comparison between single- and multi-landmark fusion strategies, while maintaining identical prediction and update formulations.

### *1.3.7 Validation*

The innovation statistics obtained for the single-landmark update are summarised as:

- Mean range innovation: −0.0010 m
- Standard deviation of range innovation: 0.0638 m
- Mean bearing innovation: 0.0469 rad
- Standard deviation of bearing innovation: 0.3038 rad

The corresponding performance indicators are illustrated in Figures 1–5.

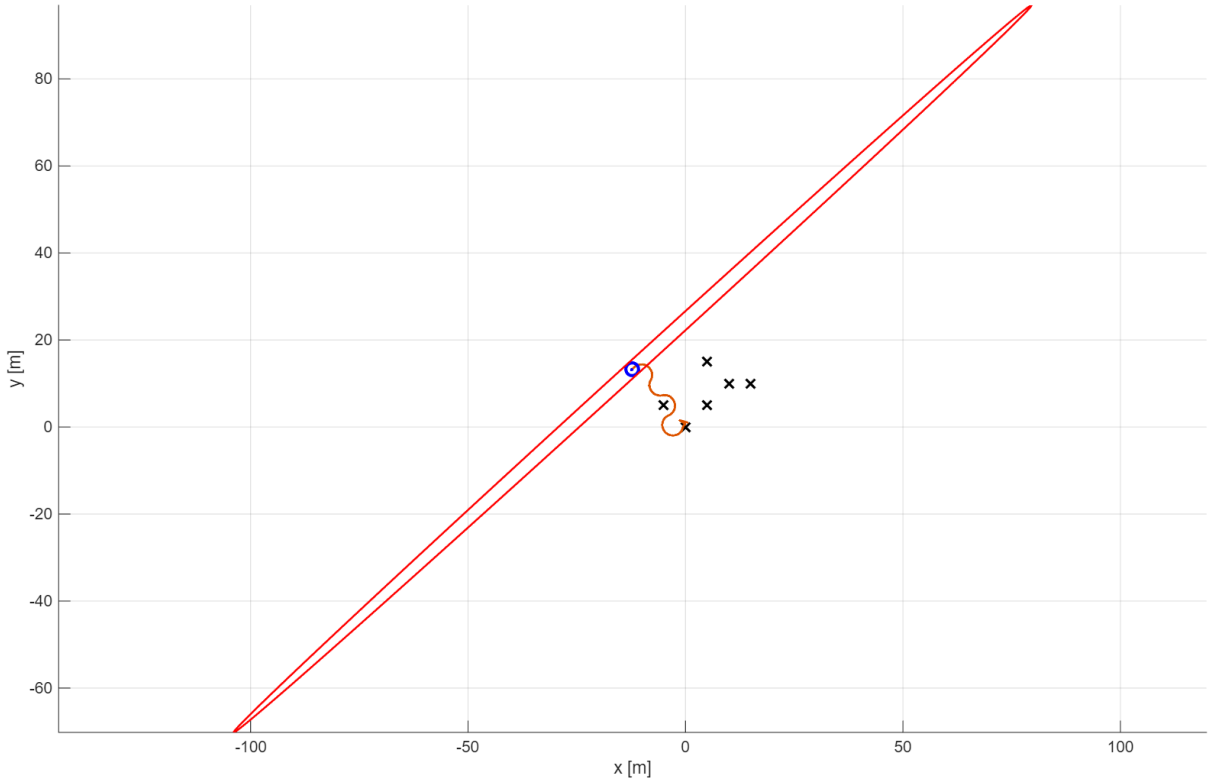


*Figure 1. Vehicle Trajectory Animation Screenshot in Single-landmark*

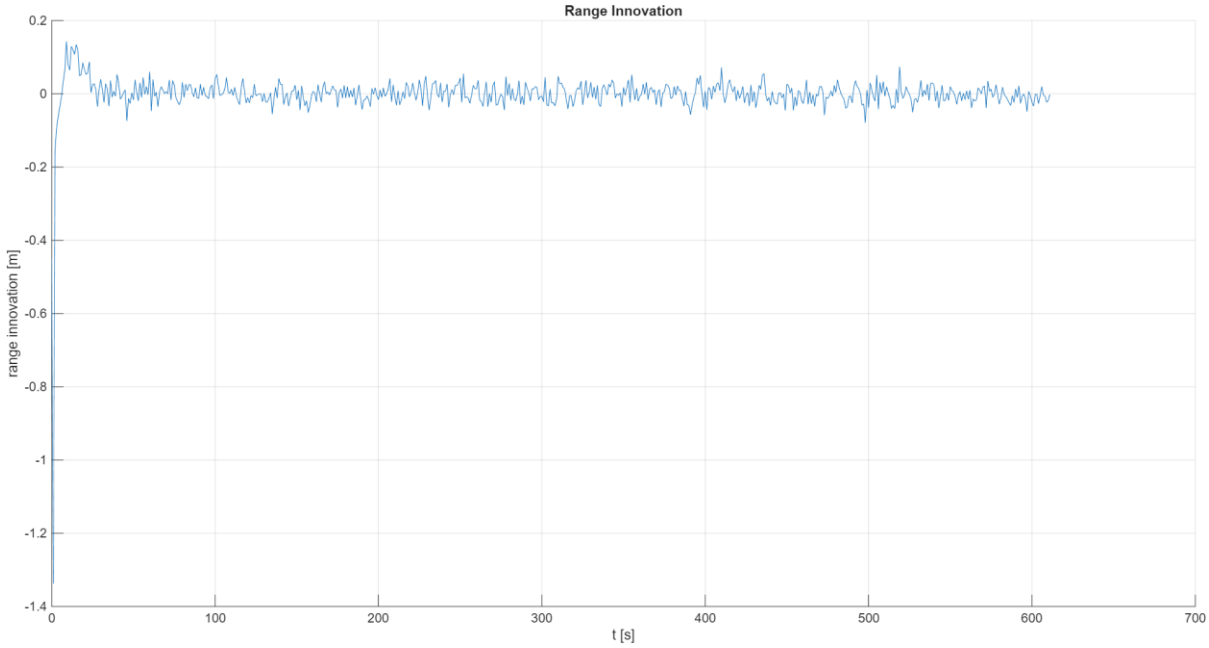


*Figure 2. Range Innovation Distribution in Single-landmark*

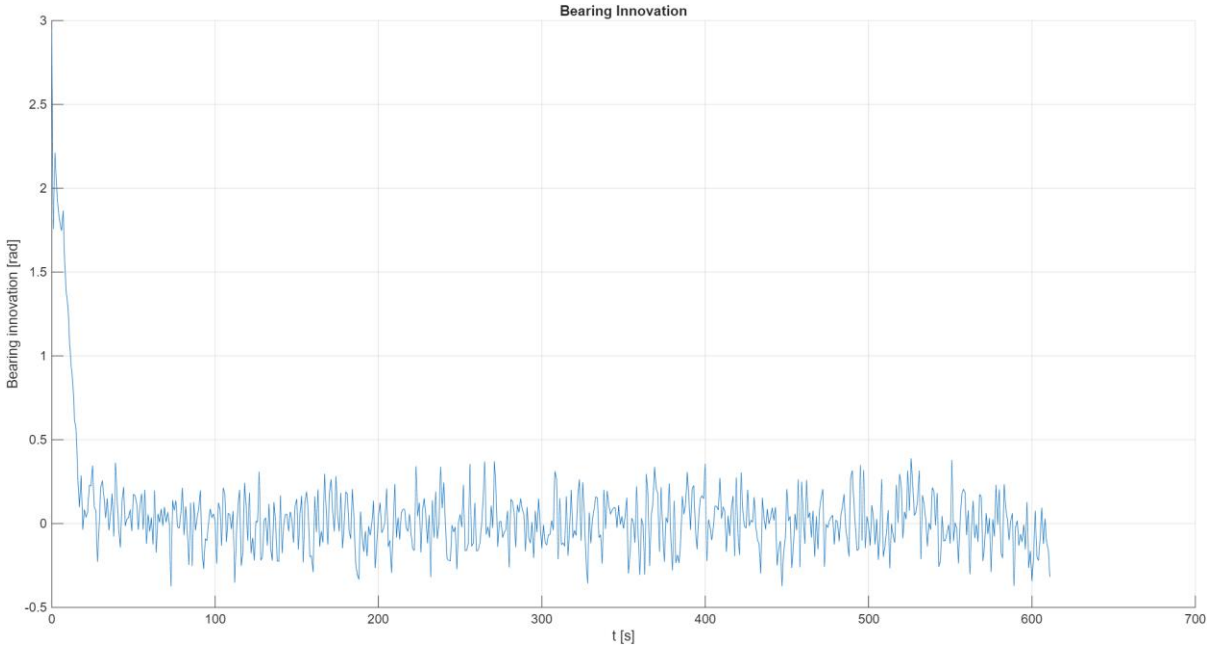


*Figure 3. Bearing Innovation Distribution in Single-landmark*

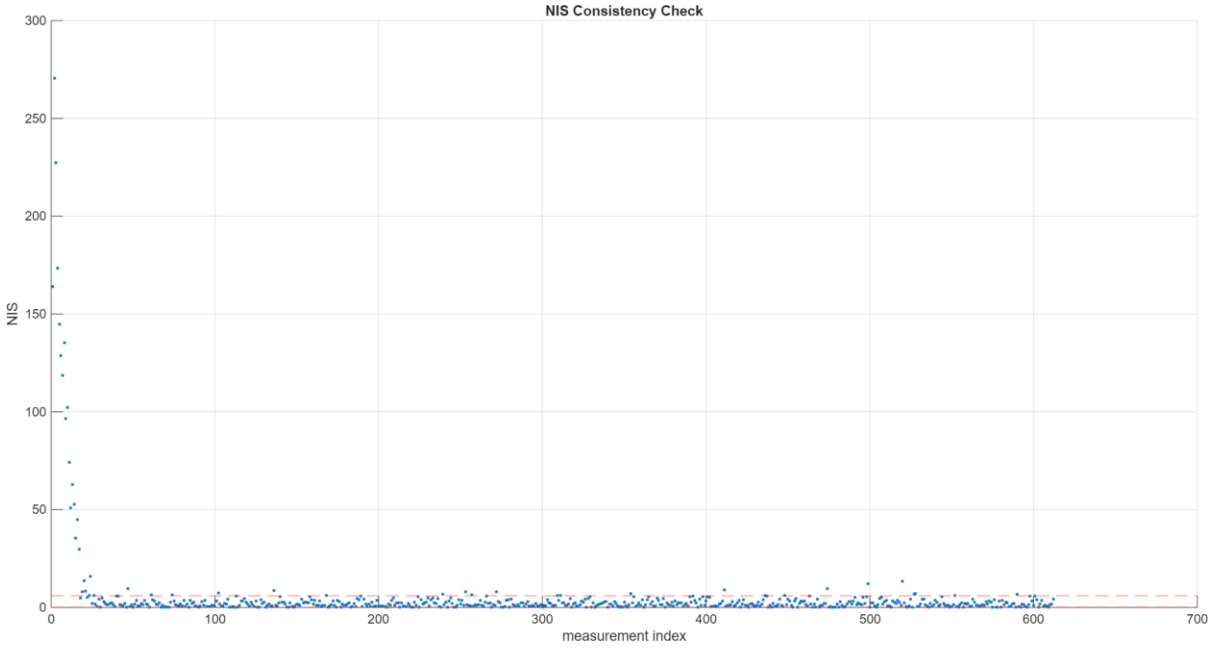


*Figure 4. NIS Consistency Check in Single-landmark*

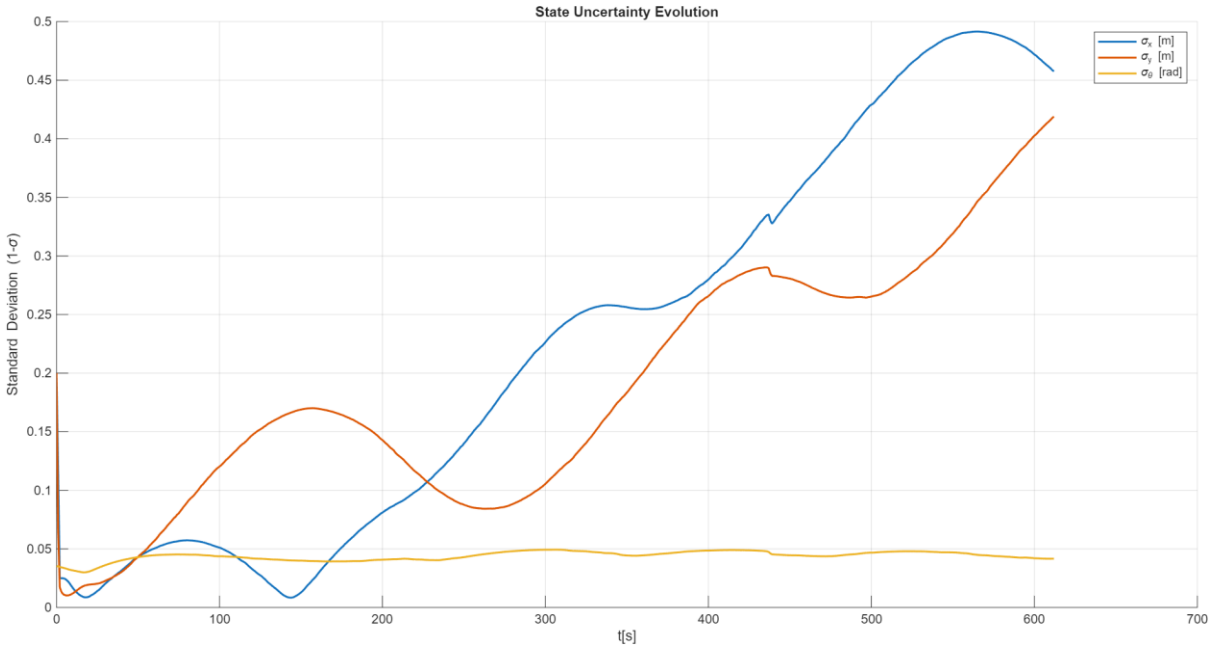


*Figure 5. State Uncertainty Evolution in Single-landmark*

A comparative discussion with the multi-landmark case is provided in Section 1.5.

## 1.4 Measurement and Update Step – Multi Landmarks

### *1.4.1 Measurement Model*

In the multi-landmark case, the same nonlinear measurement model described in Section 1.3 is applied to each observed landmark individually.

Assuming that $m$ landmarks are available at time step $k$, the overall measurement vector is constructed by stacking individual measurements:

$$\mathbf{z}_k = \begin{bmatrix} \mathbf{z}_k^1 \\ \mathbf{z}_k^2 \\ \vdots \\ \mathbf{z}_k^m \end{bmatrix}$$

Similarly, the predicted measurement is:

$$\hat{\mathbf{z}}_k^- = \begin{bmatrix} \hat{\mathbf{z}}_k^{1-} \\ \hat{\mathbf{z}}_k^{2-} \\ \vdots \\ \hat{\mathbf{z}}_k^{m-} \end{bmatrix}$$

The corresponding measurement Jacobian becomes:

$$\mathbf{H}_k = \begin{bmatrix} \mathbf{H}_k^1 \\ \mathbf{H}_k^2 \\ \vdots \\ \mathbf{H}_k^m \end{bmatrix}$$

and the measurement noise covariance matrix is constructed as a block diagonal matrix:

$$\mathbf{R}_k = \text{blkdiag}(\mathbf{R}^1, \mathbf{R}^2, \dots, \mathbf{R}^m)$$

*1.4.1 EKF Update*

The innovation vector is defined as:

$$\mathbf{y}_k = \mathbf{z}_k - \hat{\mathbf{z}}_k^-$$

The innovation covariance:

$$\mathbf{S}_k = \mathbf{H}_k \mathbf{P}_k^- \mathbf{H}_k^T + \mathbf{R}_k$$

The Kalman gain:

$$\mathbf{K}_k = \mathbf{P}_k^- \mathbf{H}_k^T \mathbf{S}_k^{-1}$$

The posterior update remains:

$$\hat{\mathbf{x}}_k = \hat{\mathbf{x}}_k^- + \mathbf{K}_k \mathbf{y}_k$$

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_k^-$$

### *1.4.3 Implementation*

In practice, the multi-landmark correction is implemented using a sequential update strategy, where each landmark measurement is incorporated one at a time within the same timestep.

Under the assumption of independent measurement noise, sequential updates are mathematically equivalent to the stacked formulation. However, sequential processing offers the following advantages:

- Reduced matrix inversion size (improving numerical stability),
- Lower computational complexity when $m$ is large,
- Improved modularity and easier debugging.

Therefore, sequential updating is adopted while maintaining theoretical equivalence to the stacked approach.

### *1.4.4 Validation*

The innovation statistics obtained for the multi-landmark update are summarised as:

- Mean range innovation: 0.0003 m
- Standard deviation of range innovation: 0.0420 m
- Mean bearing innovation: 0.0254 rad
- Standard deviation of bearing innovation: 0.2559 rad

The corresponding performance indicators are shown in Figures 6–10.

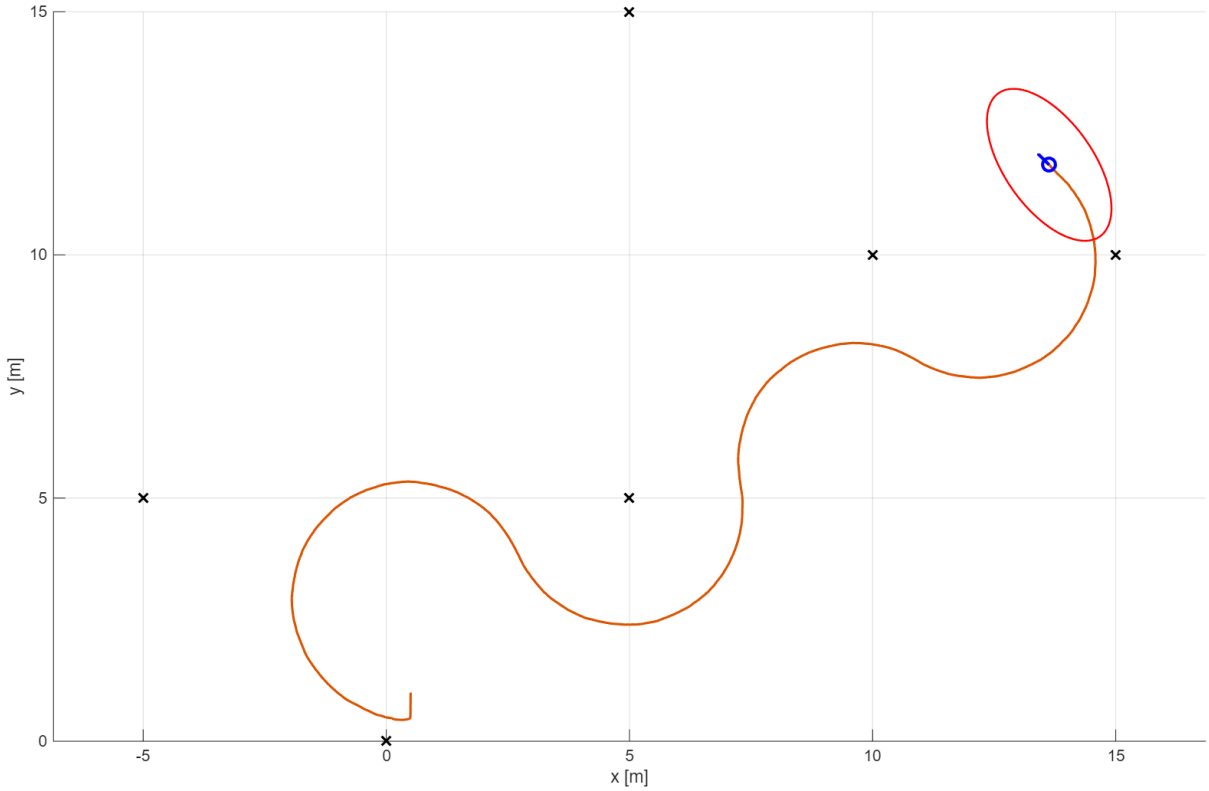


*Figure 6. Vehicle Trajectory Animation Screenshot in Multi-landmark*

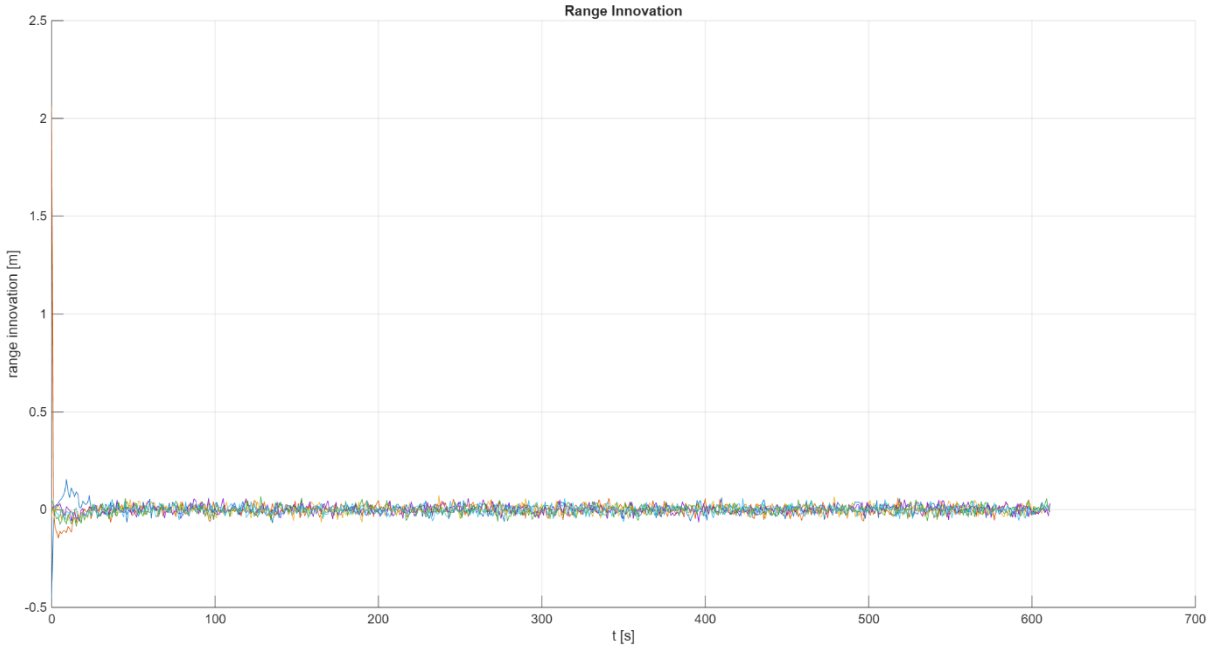


*Figure 7. Range Innovation Distribution in Multi-landmark*

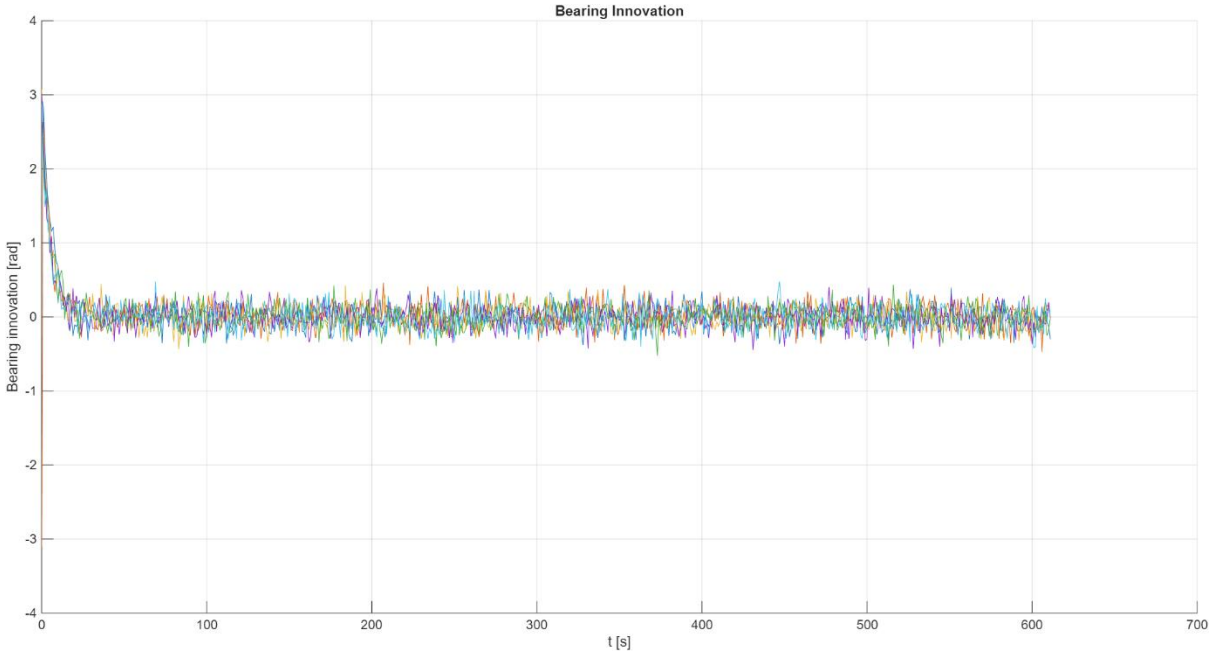


*Figure 8. Bearing Innovation Distribution in Multi-landmark*

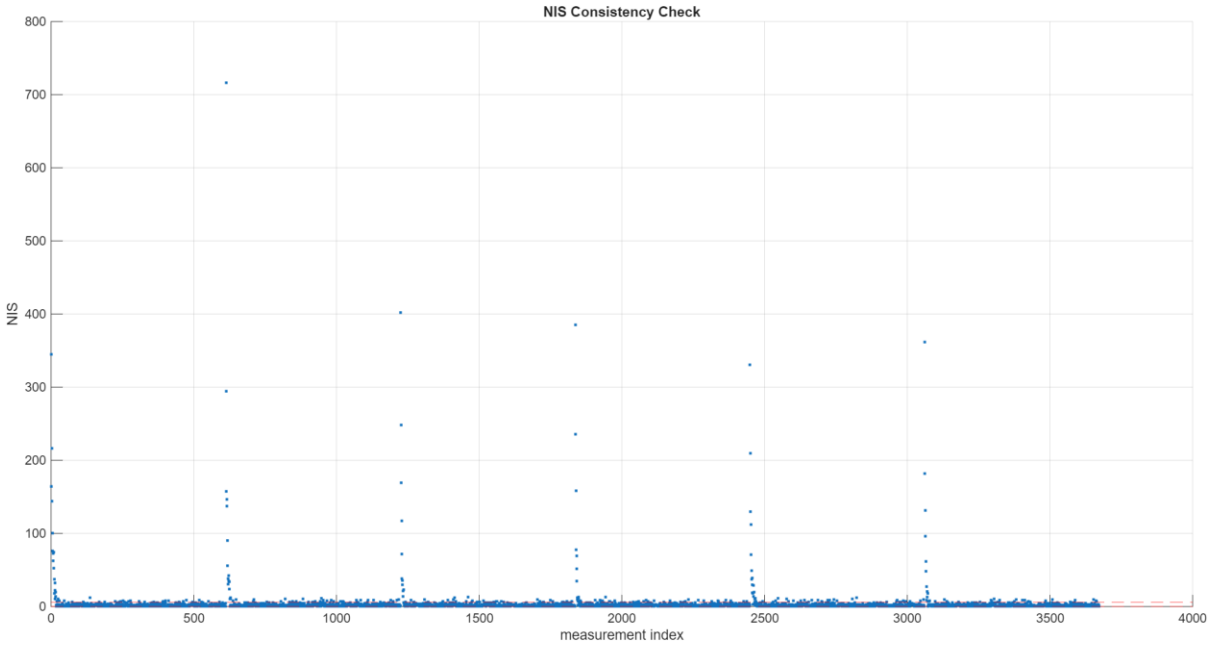


*Figure 9. NIS Consistency Check in Multi-landmark*

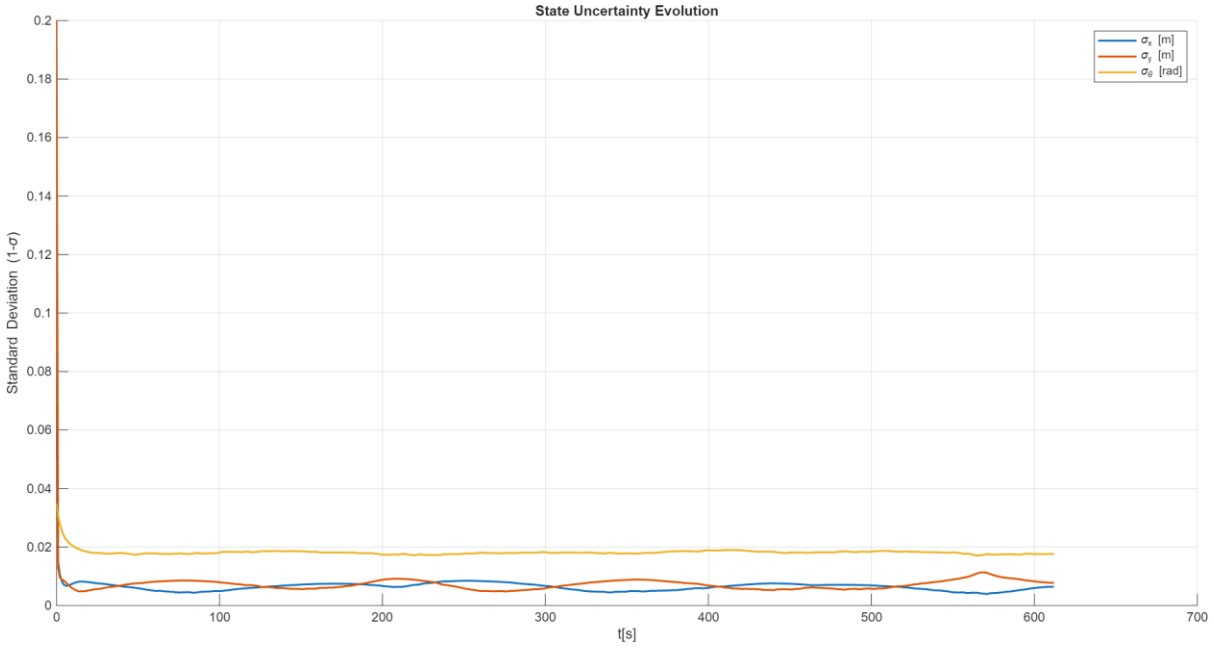


*Figure 10. State Uncertainty Evolution in Multi-landmark*

## 1.5 Comparison and Analysis

Because ground truth data are not available, direct error evaluation cannot be performed. Instead, the comparison between the single- and multi-landmark configurations is conducted in terms of statistical consistency and stability. According to the consistency definition of the Kalman filter [2],

$$E[x_k - \hat{x}_k] = 0, E[(x_k - \hat{x}_k)(x_k - \hat{x}_k)^T] = P_k,$$

where $P_k$ is the filter-predicted covariance.

From qualitative observations of the vehicle animation (Figures 1 and 6) and the state uncertainty evolution (Figures 5 and 10), clear differences can be identified. In the single-landmark configuration, the state covariance does not converge. The standard deviations of $\sigma_x$ and $\sigma_y$ show a progressive growth trend over time, and the estimated trajectory exhibits occasional abrupt corrections following measurement updates. This behaviour indicates insufficient correction capability and limited geometric constraints when only one landmark is used [3].

In contrast, under the multi-landmark configuration, the state uncertainties remain bounded and quickly stabilise within a narrow range. The trajectory becomes smoother and more stable, reflecting improved observability and stronger geometric constraints [3].

These qualitative findings are supported by the innovation statistics in Table 3. The multi-landmark approach achieves innovation means closer to zero [4] (Figures 2 and 3 versus Figures 7 and 8), with improvements of 70.0% in range and 45.8% in bearing. The standard deviations are also reduced by 34.2% and 15.8%, respectively.

| Metric | Single | Multi | Improvement |
|---|---|---|---|
| Mean Range (m) | -0.001 | 0.0003 | 70.0% ↓ (towards zero) |
| Std Range (m) | 0.0638 | 0.042 | 34.2% ↓ |
| Mean Bearing (rad) | 0.0469 | 0.0254 | 45.8% ↓ (towards zero) |
| Std Bearing (rad) | 0.3038 | 0.2559 | 15.8% ↓ |

*Table 3. Single- and multi-landmark mean and deviation comparison*

The NIS consistency check further confirms the statistical consistency of the filter, as illustrated in Figures 4 and 9. The majority of NIS values remain within the confidence bounds, indicating that the predicted innovation covariance is consistent with the observed residual statistics [5]. No persistent divergence is observed in either case.

Overall, while both configurations remain statistically consistent, the multi-landmark update significantly enhances stability, reduces innovation variance, and improves state observability compared to the single-landmark solution.

### 1.6 Additional Experiment for Timestep Consistency

In the assignment specification, the nominal timestep is defined as $T = 0.1\ \mathrm{s}$. However, the time vector provided in my_input.mat is sampled at 1-second intervals. This difference between the nominal model timestep and the dataset sampling interval may introduce dimensional inconsistency in the state propagation step.

To investigate the impact of timestep selection, an additional experiment was conducted under the same multi-landmark configuration. The baseline case uses $T = 1\ \mathrm{s}$, while the experimental case applies $T = 0.1\ \mathrm{s}$ without rescaling the dataset.

The results for $T = 0.1\ \mathrm{s}$ are shown in Figures 11–15.

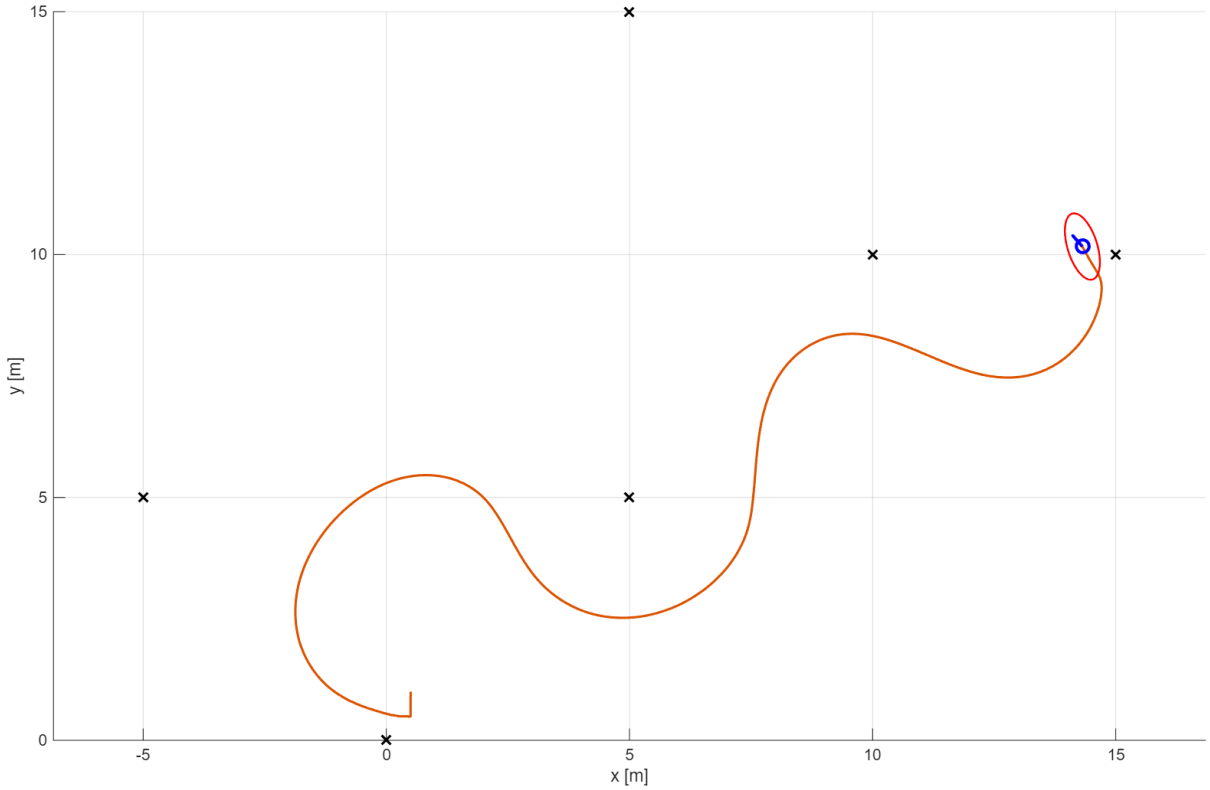


*Figure 11. Vehicle Trajectory Animation Screenshot in 0.1 Timestep*

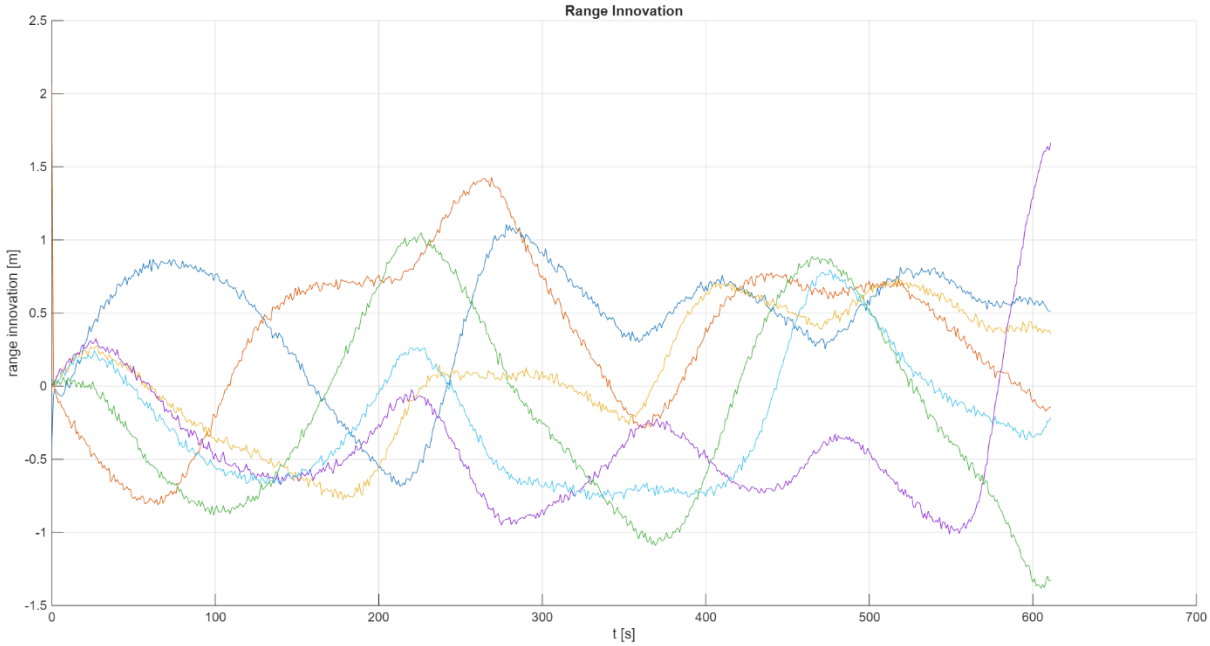


*Figure 12. Range Innovation Distribution in 0.1 Timestep*

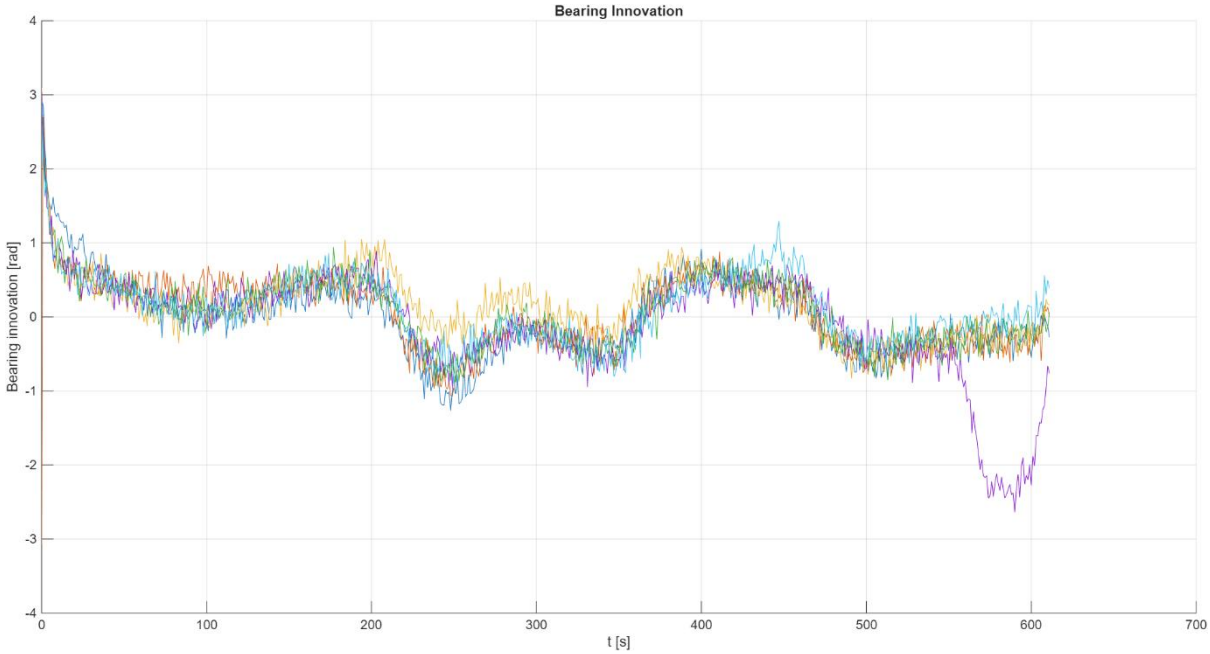


*Figure 13. Bearing Innovation Distribution in 0.1 Timestep*

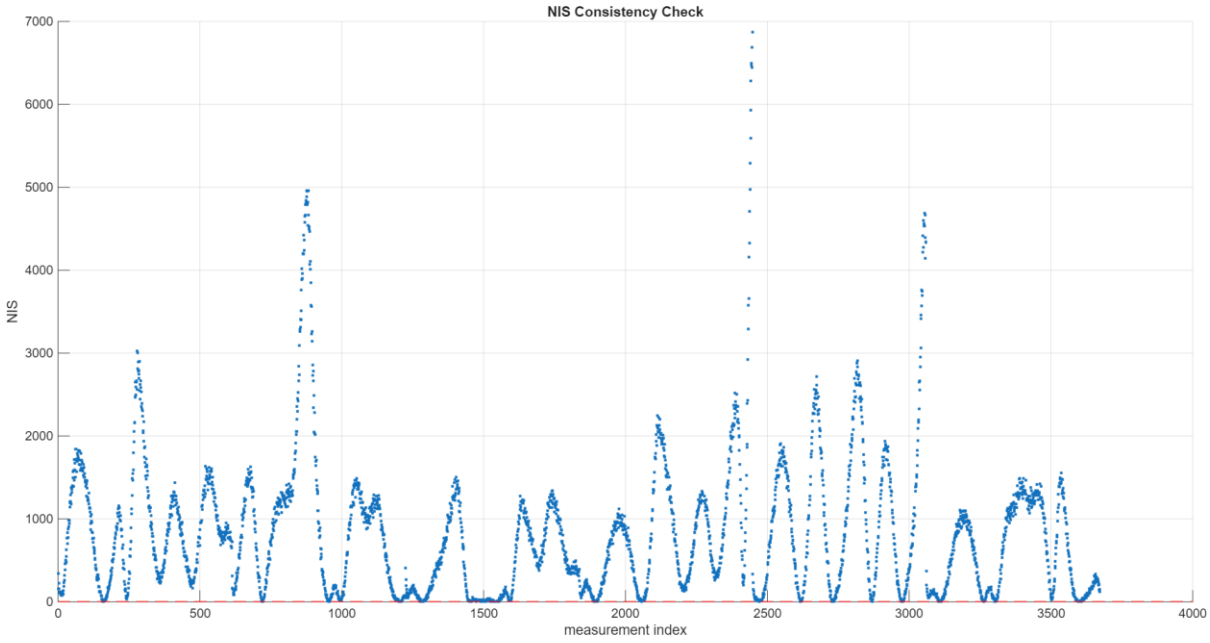


*Figure 14. NIS Consistency Check in 0.1 Timestep*

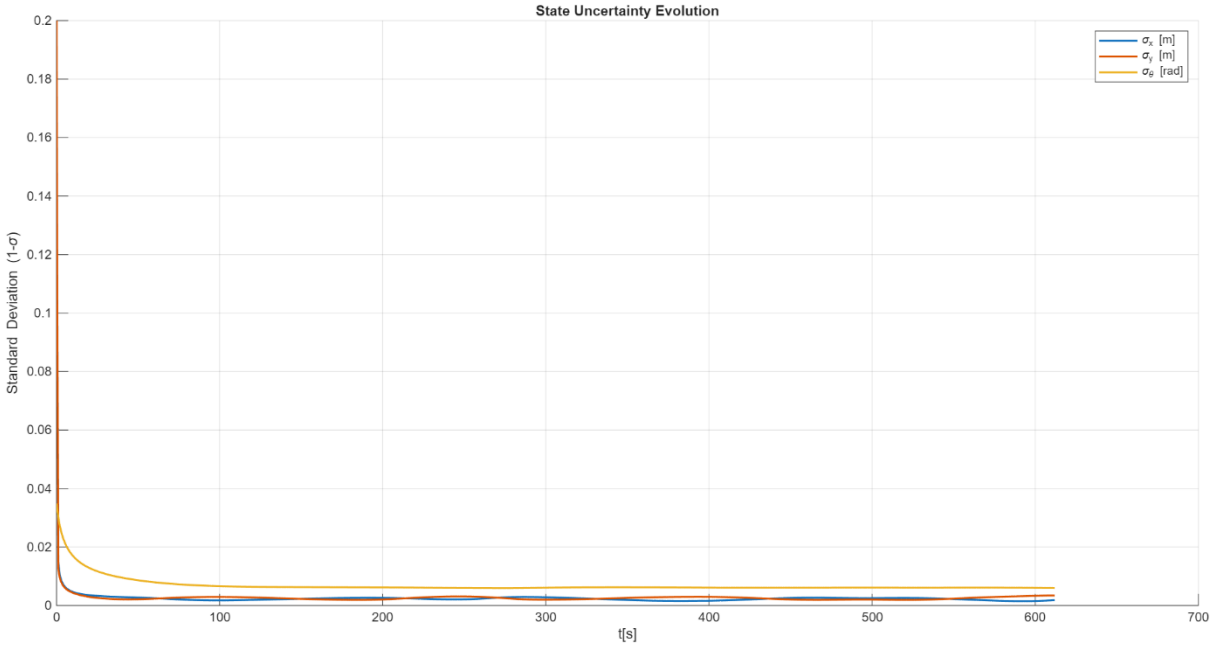


*Figure 15. State Uncertainty Evolution in 0.1 Timestep*

Although the trajectories appear visually similar (Figure 11), the innovation behaviour reveals significant differences. Under $T = 0.1\ s$, the range and bearing innovations (Figures 12–13) exhibit increased variance, and the NIS values (Figure 14) frequently exceed the confidence bounds, indicating statistical inconsistency.

This behaviour arises because the state propagation increment and associated process noise scaling become inconsistent with the actual sampling interval of the odometry data. Consequently, the filter overestimates state changes relative to the measurement update frequency, leading to divergence-like behaviour in the innovation statistics.

These results confirm that using $T = 1\ s$, consistent with the dataset sampling interval, ensures model–data consistency and statistically stable filtering performance.

# 2. Task 2 – Path Planning and Following

## 2.1 Task 2.1 – Dubins Path

### *2.1.1 Design a Dubins Path*

| Seg | Waypoints (x,y, θ°) Start → End | Circle Centres (Start / End) | Exit angles φ_ex (deg) | Entry angles φ_en (deg) | Exit Location T_X (m) | Entry Location T_N (m) |
|---|---|---|---|---|---|---|
| 1 | (0,10,0°) → (60,60,45°) | (0,15) / (56.46,63.54) | -49.32 | -49.32 | (3.26,11.21) | (59.72,59.74) |
| 2 | (60,60,45°) → (80,120,30°) | (56.46,63.54) / (82.50,115.67) | -16.66 | 163.34 | (61.25,62.10) | (77.71,117.10) |
| 3 | (80,120,30°) → (150,70,-90°) | (82.50,115.67) / (145,70) | 53.84 | 53.84 | (85.45,119.71) | (147.95,74.04) |
| 4 | (150,70,-90°) → (100,30,-120°) | (145,70) / (104.33,27.50) | -53.53 | 126.47 | (147.97,65.98) | (101.36,31.52) |
| 5 | (100,30,-120°) → (50,10,-180°) | (95.67,32.50) / (50,15) | -69.03 | -69.03 | (97.46,27.83) | (51.79,10.33) |

*Table 4. Dubins Path Design Key Parameters*

### *2.1.2 Path Drawing*

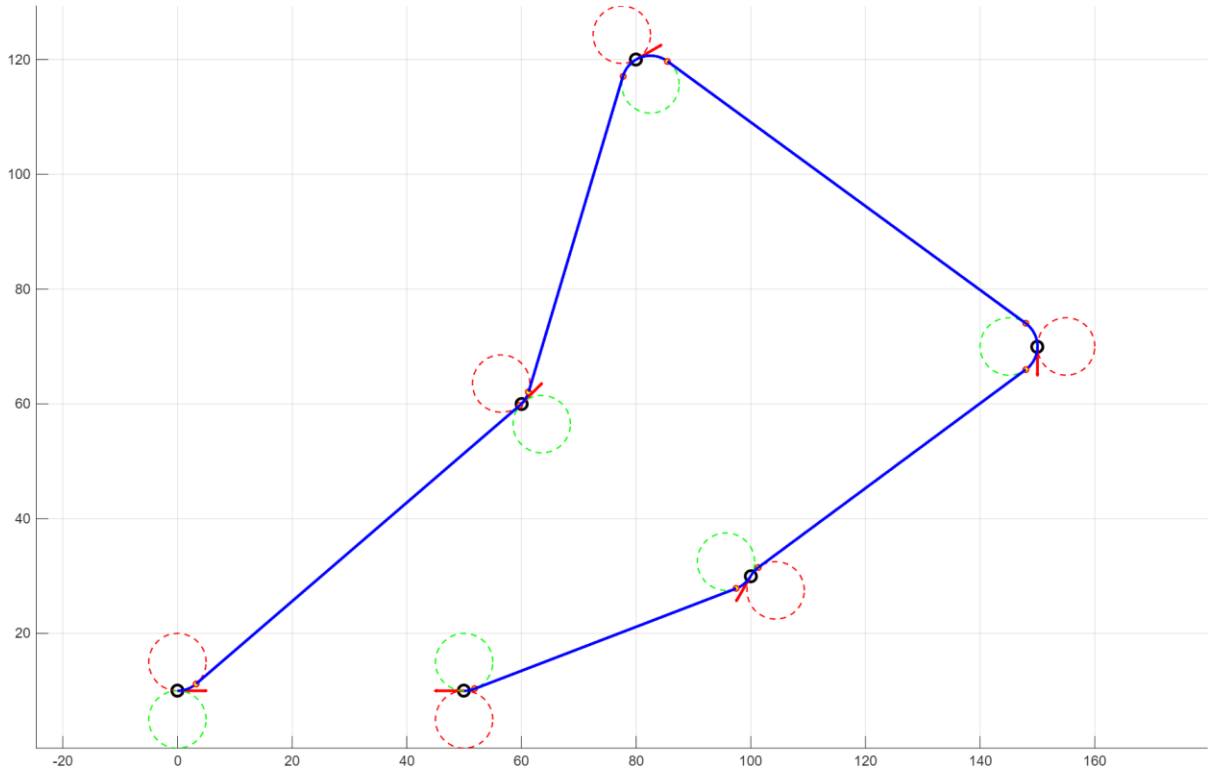


*Figure 16. Dubins Path Drawing*

## 2.2 Task 2.1 – CCA Algorithm

### *2.3.1 Describe Carrot-chasing Algorithm*

The Carrot-Chasing Algorithm (CCA) is a geometric path-following method commonly used for mobile robots and autonomous vehicles. Instead of directly tracking the reference path, the algorithm guides the vehicle towards a virtual target point located ahead of the vehicle along the desired path. By continuously updating this target point during motion, the vehicle gradually converges to the reference path and maintains stable tracking.

At each control step, the current vehicle position is first projected onto the reference path to determine the closest point on the path. A virtual target point (VTP) is then placed ahead of this projection using a predefined look-ahead distance. The vehicle is commanded to move towards this virtual point rather than the projection point itself. This forward-looking behaviour prevents oscillations and improves tracking stability.

The desired heading direction is computed as the angle between the vehicle position and the virtual target point

$$\psi_d = \text{atan2}\,(y_t - y,\, x_t - x)$$

where $(x, y)$ represents the vehicle position and $(x_t, y_t)$ denotes the virtual target point.

The heading error is calculated as

$$e_\psi = \text{wrapToPi}\,(\psi_d - \psi)$$

where $\psi$is the current vehicle heading.

A curvature-based control law is then used to generate steering commands

$$\kappa = k_\psi e_\psi$$

where $k_\psi$ is the heading gain. The angular velocity of the vehicle can therefore be expressed as

$$\omega = v\kappa$$

where $v$is the forward velocity.

By continuously updating the virtual target point and correcting the heading error, the vehicle progressively converges to the desired path. The look-ahead distance plays a critical role in the behaviour of the algorithm. A small look-ahead distance improves tracking accuracy but may lead to oscillations, while a larger look-ahead distance produces smoother motion at the cost of increased tracking error.

*2.3.2 Carrot-chasing Implementation*

To follow the Dubins path generated in Question 1, a segment-based carrot-chasing simulation was implemented in MATLAB. Since each Dubins path consists of an initial circular arc, a straight tangent segment, and a final circular arc, the controller was organised using a simple state-machine structure to ensure sequential progression through these three sub-paths.

For the straight-line segment, the vehicle position is projected onto the tangent line, and a virtual target point is placed ahead of the projection using a fixed look-ahead distance of 5 m. The desired heading is then defined as the line-of-sight direction from the vehicle to this virtual target point.

For the circular arc segments, the vehicle progress is estimated from its angular position relative to the arc centre. A carrot point is then generated ahead of the current arc progress using an angular look-ahead of 0.5 rad. To improve tracking behaviour on curved segments, the desired heading is not taken purely as the line-of-sight direction to the carrot point. Instead, it is computed as a weighted combination of the tangent direction at the carrot point and the line-of-sight direction from the vehicle to the carrot point. This helps the vehicle align with the path direction while still converging towards the target point.

The heading error is defined as

$$e_\psi = \text{wrapToPi}\ (\psi_d - \psi)$$

and the angular velocity command is generated using a proportional heading law

$$\omega = v k_\psi e_\psi$$

where $v = 5m/s$ is the forward velocity and $k_\psi = 5.0$ is the heading gain. The angular velocity is further limited according to the maximum lateral acceleration constraint, such that the steering command remains dynamically feasible.

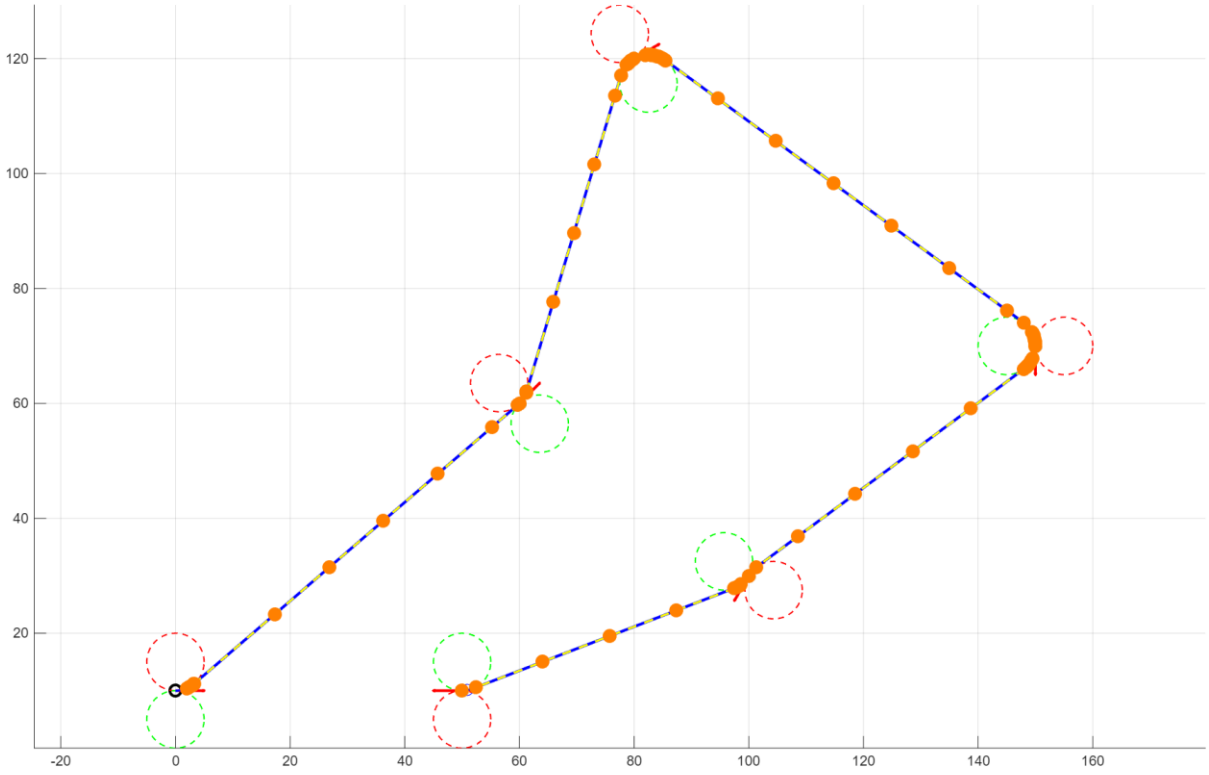


*Figure 17. Carrot Chasing Performance*

Simulation results, as illustrated in Figure 17, show that the vehicle can smoothly track both straight and curved sections of the Dubins path, while maintaining continuous progression through multiple path segments. The full MATLAB implementation is provided in Appendix 02.

### 2.3.3 Carrot-chasing Parameter Discussion

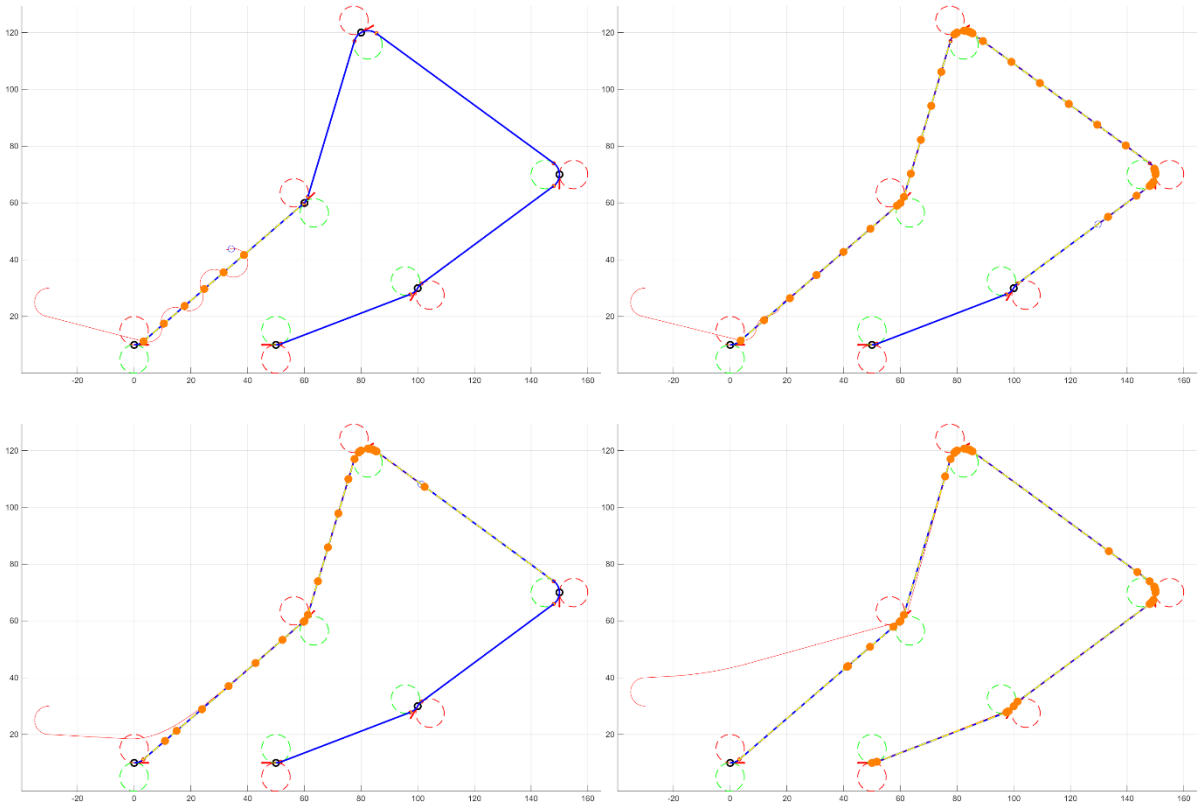

*Figure 18. Different Performance in δ = 0.0, 0.5, 10, 50*

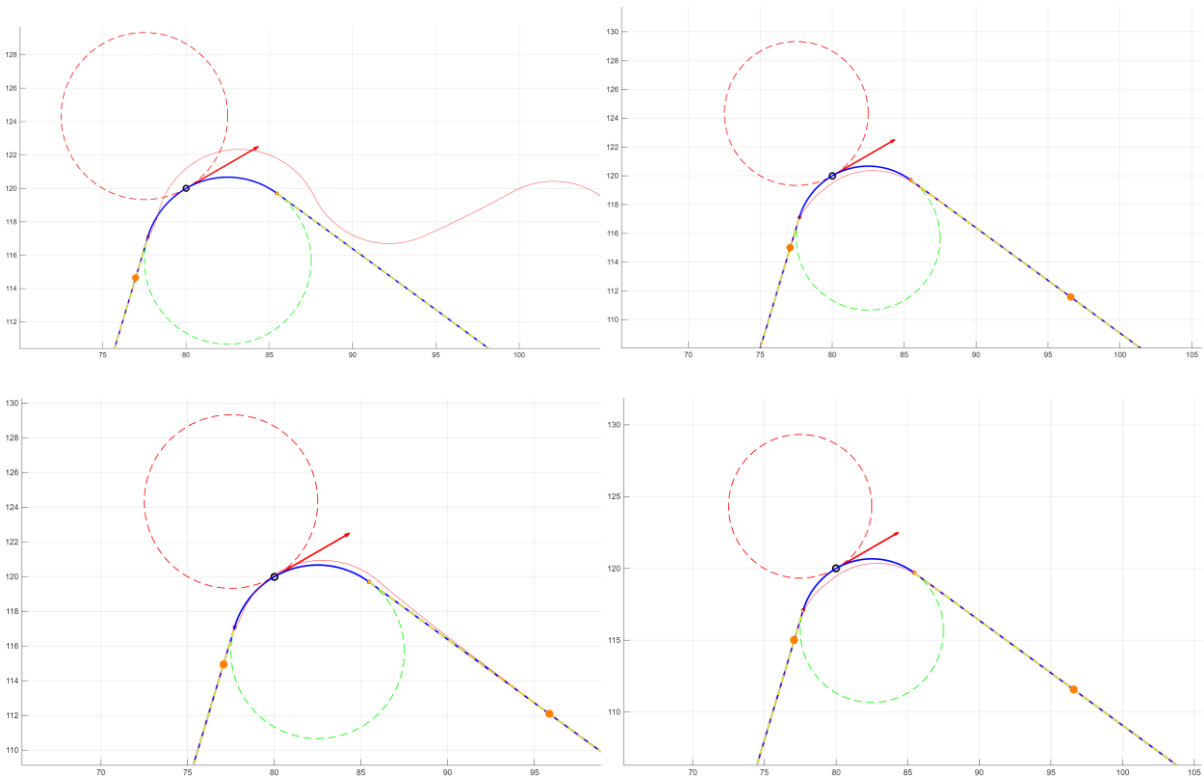

*Figure 19. Different Performance in λ = 0.0, 0.3, 0.5, 1.0*

The main tuning parameters of the carrot-chasing algorithm are the look-ahead distance $\delta$for straight-line tracking and the angular look-ahead parameter $\lambda$for circular tracking.

For straight-line tracking, the influence of $\delta$is illustrated in Figure 18, where $\delta$is varied from small to large values. When $\delta$is very small (e.g., 0.0 m or 0.5 m), the vehicle exhibits an S-shaped oscillatory trajectory around the reference line. This behaviour results from aggressive heading corrections, as the virtual target point is located very close to the vehicle, causing frequent changes in steering direction and overshoot. As $\delta$increases, the trajectory becomes smoother and the convergence to the desired straight line is more stable. However, excessively large values of $\delta$reduce responsiveness, leading to slower convergence toward the path [8].

For circular tracking, the influence of the angular look-ahead parameter $\lambda$is shown in Figure 19. When $\lambda = 0$, the vehicle fails to maintain the desired circular trajectory and gradually deviates from the reference path. This occurs because the look-ahead point coincides with the projection point, providing insufficient curvature anticipation. Increasing $\lambda$ to moderate values (e.g., 0.3, 0.5 or 1.0) significantly improves tracking performance, as the controller better anticipates the required curvature. However, different values of $\lambda$ lead to slightly different tracking paths due to variations in curvature compensation [8].

Overall, excessively small look-ahead parameters increase control aggressiveness and may induce oscillatory behaviour. Straight-line tracking is mainly affected in terms of overshoot and oscillation when $\delta$is small, whereas circular tracking is more sensitive to $\lambda$, since insufficient angular anticipation directly results in curvature mismatch and sustained deviation from the desired circular path.

## 2.3 Task 2.2 – RRT Algorithm

### *2.4.1 RRT Algorithm*

The Rapidly-exploring Random Tree (RRT) [9] incrementally constructs a tree $T$ rooted at the start configuration $p_{\text{start}}$ in order to explore the collision-free workspace. At each iteration, a random configuration is uniformly sampled within the map boundaries such that

$$x_{\text{rand}} \sim \mathcal{U}(x_{\min}, x_{\max}), y_{\text{rand}} \sim \mathcal{U}(y_{\min}, y_{\max}),$$

forming the sample point $p_{\text{rand}} = (x_{\text{rand}}, y_{\text{rand}})$.

The nearest existing node in the tree is then selected according to the Euclidean metric,

$$p_{\text{near}} = \arg\min_{p_i \in T} \| p_{\text{rand}} - p_i \|_2,$$

where

$$\| p_{\text{rand}} - p_i \|_2 = \sqrt{(x_r - x_i)^2 + (y_r - y_i)^2}.$$

To expand the tree, a new node is generated by steering from $p_{\text{near}}$ toward $p_{\text{rand}}$ using a fixed step size Δ. The normalized direction vector is defined as

$$\hat{d} = \frac{p_{\text{rand}} - p_{\text{near}}}{\| p_{\text{rand}} - p_{\text{near}} \|},$$

and the new configuration is obtained as

$$p_{\text{new}} = p_{\text{near}} + \Delta \hat{d}.$$

The node $p_{\text{new}}$ is added to the tree only if the connecting segment is collision-free. The procedure continues iteratively until the generated node satisfies the goal condition

$$\| p_{\text{new}} - p_{\text{goal}} \| < r_g,$$

where $r_g$ denotes the goal tolerance. Once this condition is met, the feasible path is extracted by backtracking from the goal-reaching node to the root using the stored parent relationships.

### *2.4.2 Code Implementation*

The RRT framework described above was implemented in MATLAB within the +path_planning/rrt/rrt.m script. The algorithm follows the standard procedure of uniform random sampling, nearest-neighbour selection, fixed-step steering, and collision checking to incrementally construct a feasible exploration tree in the configuration space.

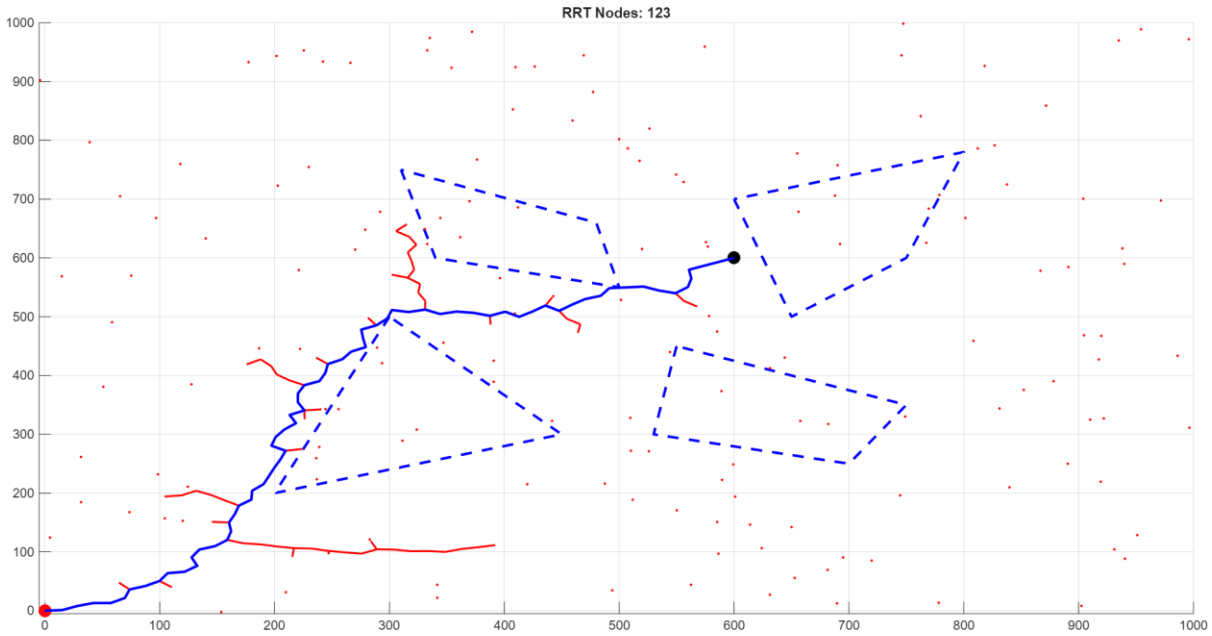


*Figure 20. RRT Path Generation and Path Selection*

As shown in Figure 20, RRT successfully generates a collision-free path between the start and goal poses. However, due to its stochastic nature, the resulting path typically contains unnecessary detours and sharp heading variations, making it unsuitable for curvature-constrained vehicles.

To improve path quality, a two-stage post-processing strategy was adopted. First, a shortcut smoothing procedure was applied, where pairs of orderly selected waypoints were directly connected if the straight segment was collision-free, thereby reducing path redundancy (Figure 21).

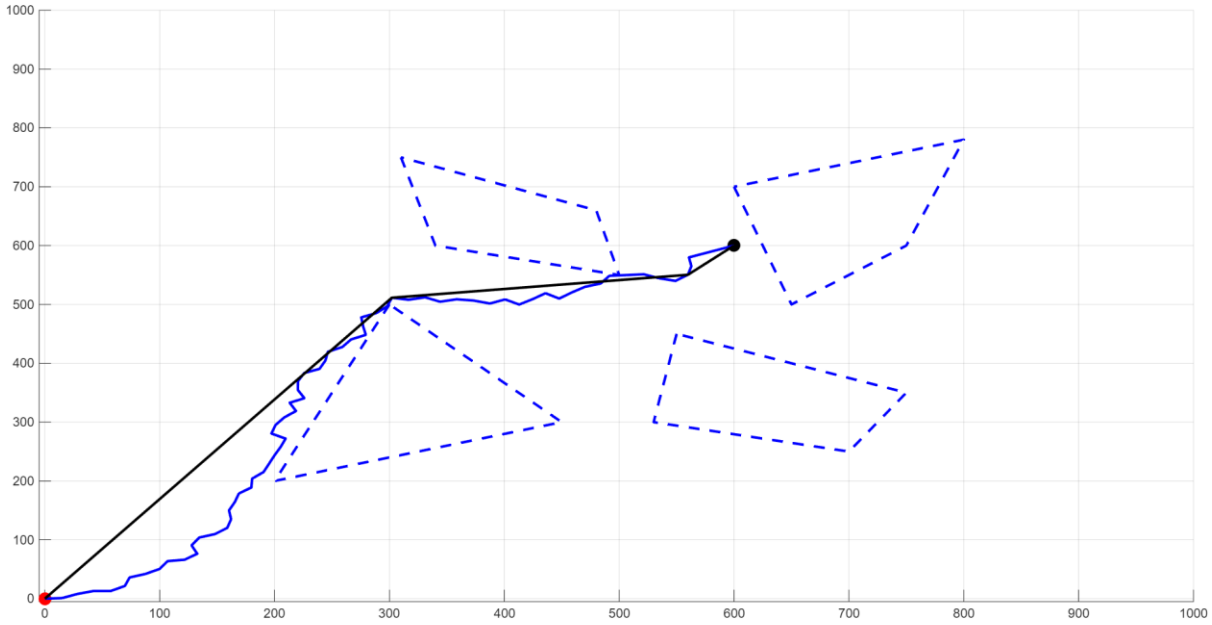


*Figure 21. RRT Path Shortcut Optimization*

Second, Dubins curves were generated between consecutive waypoints to enforce minimum turning radius constraints and ensure curvature feasibility (Figures 22–23). This guarantees geometric consistency with the vehicle's kinematic limitations.

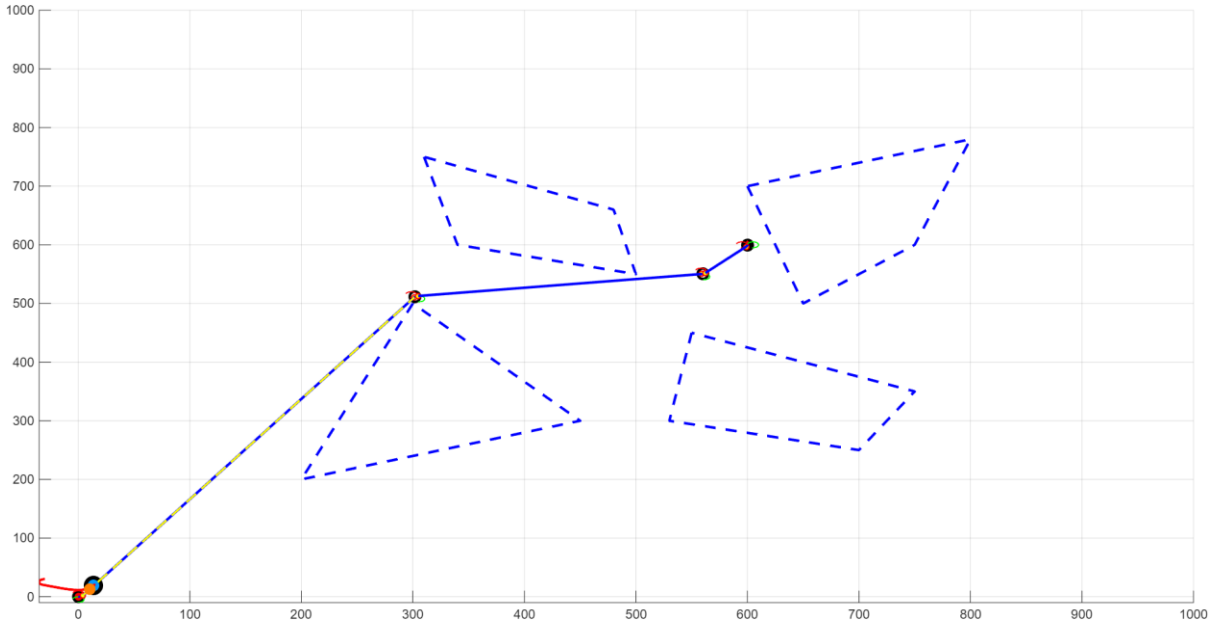


*Figure 22. Dubins Path Generation*

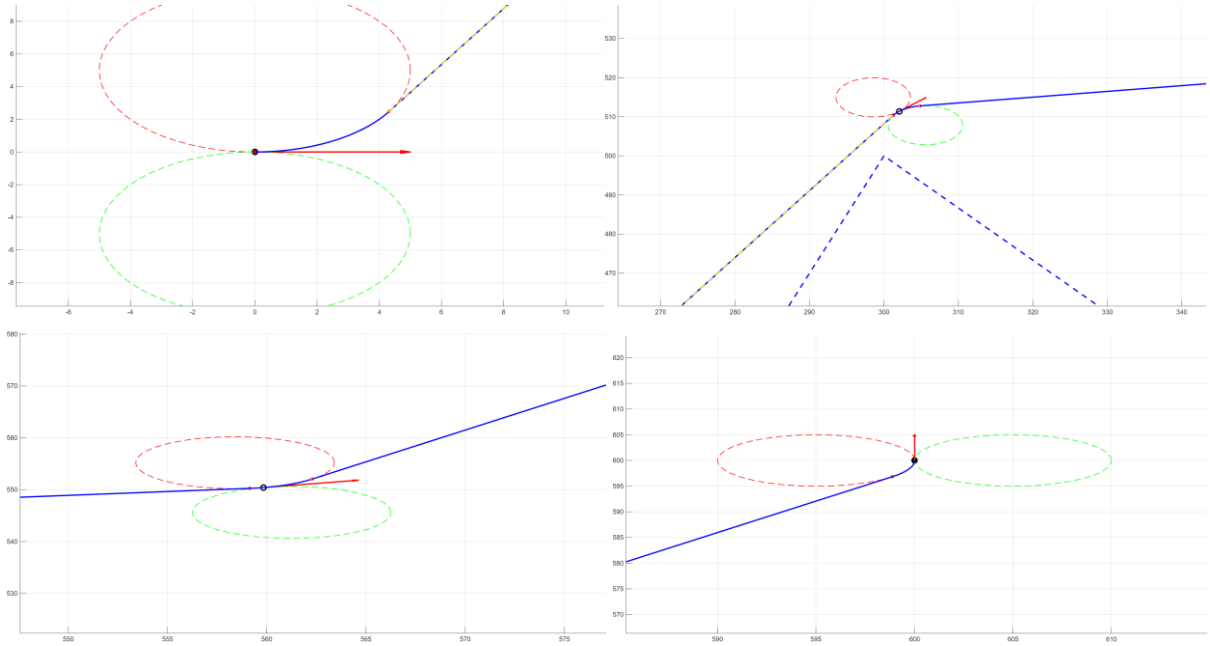

*Figure 23. Dubins Path Waypoint Specification*

Finally, the refined Dubins path was tracked using the carrot-chasing controller, resulting in smooth convergence to the target pose (Figure 24).

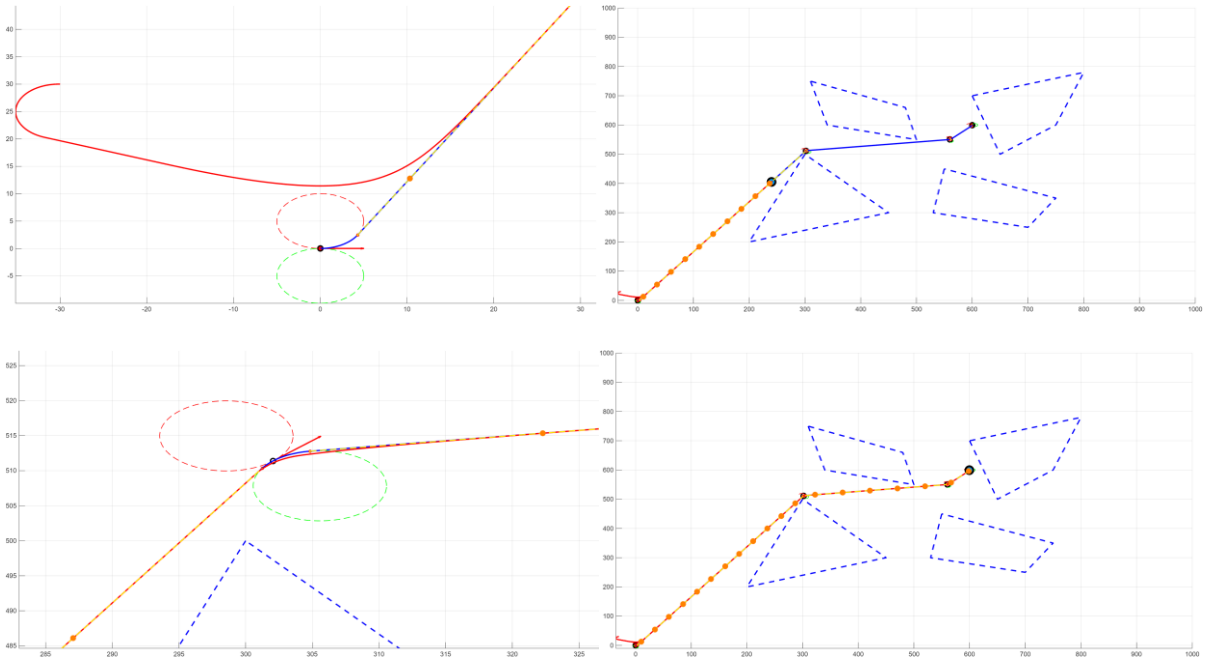

*Figure 24. Carrot Chasing Performance*

The overall framework integrates global exploration (RRT), geometric optimisation (shortcut and Dubins refinement), and local tracking control (carrot-chasing), forming a complete autonomous navigation pipeline consistent with the principles studied in the Autonomy course.

The draw-polygon function and collision-checking routine was adopted from the provided coursework template to ensure consistency with the assignment environment.

## References


1. Welch G, Bishop G. An introduction to the Kalman filter [Internet]. Chapel Hill (NC): University of North Carolina at Chapel Hill; 1997 [cited 2026 Feb 24]. Available from: https://www.cs.unc.edu/~welch/media/pdf/kalman_intro.pdf
2. Jwo D-J, Cho T-S. A practical note on evaluating Kalman filter performance optimality and degradation. Applied Mathematics and Computation [Internet]. 2007 [cited 2026 Feb 24];193(2):482–505. Available from: https://doi.org/10.1016/j.amc.2007.04.008
3. Huang G, Mourikis AI, Roumeliotis SI. Observability-based rules for designing consistent EKF SLAM estimators. The International Journal of Robotics Research [Internet]. 2010 [cited 2026 Feb 24];29(5):502–528. Available from: https://doi.org/10.1177/0278364909353640
4. Bar-Shalom Y, Li XR, Kirubarajan T. Estimation with applications to tracking and navigation: theory, algorithms and software. Hoboken (NJ): John Wiley & Sons; 2001.
5. Ragab H, Khamis S, Napoleon SA. Advanced object tracking in self-driving cars: EKF and UKF performance evaluation. In: 2024 Novel Intelligent and Leading Emerging Sciences Conference (NILES) [Internet]; 2024; Cairo. Piscataway (NJ): IEEE; 2024 [cited 2026 Feb 24]. Available from: https://doi.org/10.1109/NILES63360.2024.10753230
6. Shanmugavel M. Path planning of multiple autonomous vehicles [PhD thesis on the Internet]. Blacksburg (VA): Virginia Polytechnic Institute and State University; 2007 [cited 2026 Feb 24]. Available from: http://hdl.handle.net/1826/1745
7. Park S, Deyst J, How JP. A new nonlinear guidance logic for trajectory tracking. In: AIAA Guidance, Navigation, and Control Conference and Exhibit [Internet]; 2004 Aug 16–19; Providence (RI). Reston (VA): AIAA; 2004 [cited 2026 Feb 24]. Available from: https://doi.org/10.2514/6.2004-4900
8. Coulter RC. Implementation of the pure pursuit path tracking algorithm [Internet]. Pittsburgh (PA): Carnegie Mellon University, Robotics Institute; 1992 [cited 2026 Feb 24]. Available from: https://apps.dtic.mil/sti/html/tr/ADA255524/
9. Kothari M, Postlethwaite I, Gu D-W. A suboptimal path planning algorithm using rapidly-exploring random trees. International Journal of Aerospace Innovations [Internet]. 2010 [cited 2026 Feb 24];2(1–2):93–104. Available from: https://doi.org/10.1260/1757-2258.2.1-2.93

# Appendices

## 01_ Programming of Task 1 – Robot Tracking with Non-Linear Filter

### main.m

```
clc; clear; close all;


%% ------------------------------------------------------------
%  Control Input Dataset
%
%  v(k)   : Linear velocity           [m/s]
%  om(k)  : Angular velocity          [rad/s]
%  t(k)   : Discrete time sequence    [s]
%
%  The control input vector is defined as:
%     u(k) = [v(k); om(k)]
% -------------------------------------------------------------
load("data\my_input.mat");


%% ------------------------------------------------------------
%  Measurement Dataset
%
%  r(k)   : range measurement           [m]
%  b(k)   : bearing measurement         [rad]
%  l(k)   : position of the 6 landmarks in X,Y coordinates   [m]
%
%  The Measurement vector is defined as:
%     z(k) = [r(k); b(k)]
% -------------------------------------------------------------
load("data\my_measurements.mat");


%% ============================================================
```

```
%  Data Sanity Check
% =================================================================
fprintf('r range: min=%.3f max=%.3f mean=%.3f\n', min(r(:),[],'omitnan'), max(r(:),[],'omitnan'), mean(r(:),'omitnan'));
fprintf('b range: min=%.3f max=%.3f mean=%.3f\n', min(b(:),[],'omitnan'), max(b(:),[],'omitnan'), mean(b(:),'omitnan'));
fprintf('max |b| = %.3f\n', max(abs(b(:)),[],'omitnan'));


%% =================================================================
%  EKF INITIALISATION
% =================================================================


% ----- State -----
x_pos = 0.5;    % [m]
y_pos = 1;      % [m]
theta = 0;      % [rad]
x = [x_pos; y_pos; theta];


% ----- lidar distance respect to robot centre -----
d = 0; % [m]


% ----- Timestep -----
dt = 1;      % [s]


% ----- Number of control steps -----
N_u = length(v);


% ----- Number of state -----
N_x = N_u + 1;


% ----- Process Noise Covariance Matrix (Q) -----
std_lin_vel = 0.007;     % [m/s]
```

```
std_ang_vel = 0.0075;    % [rad/s]
Q = diag([std_lin_vel^2, std_ang_vel^2]);

% ----- Measurement Noise Covariance Matrix (R) -----
std_range = 0.02;       %[m]
std_bearing = 0.15;     %[rad]
R = diag([std_range^2, std_bearing^2]);

% ----- State Error Covariance Matrix (P) -----
std_x0 = 0.2;           %[m]
std_y0 = 0.2;           %[m]
std_theta0 = deg2rad(2);  %[rad]
P = diag([std_x0^2, std_y0^2, std_theta0^2]);
P_row = size(P, 1);
P_col = size(P, 2);
P_hist = nan(P_row, P_col, N_x);
P_hist(:,:,1) = P;

% ----- Cavans and Information Print  -----
x_hist = zeros(3, N_x);
x_hist(:,1) = x;

% ----- Innovation Analysis  -----
innovation = [nan; nan];
LM_row = size(l, 1);
r_inno_hist = nan(N_u, LM_row);
b_inno_hist = nan(N_u, LM_row);
mea_dimension = size(innovation, 1);
chi_confi_range = 0.95;

% ----- NIS(Normalized Innovation Square) Analysis  -----
```

```
NIS_hist = nan(N_u, LM_row);

% ----- Single or Muti Landmark  -----
single_mode_enabled = true;
if single_mode_enabled
   LM_row = 1;
else
   LM_row = size(l, 1);
end

%% ================================================================
%  EKF Main Loop
% ================================================================
for k = 1:N_u % 1 - 612

   % Get control input from dataset by timestep
   v_k = v(k);
   om_k = om(k);

   % 1. State Prediction - Prior State (x_pri)
   x_pri = [
      x(1) + dt * cos(x(3)) * v_k;
      x(2) + dt * sin(x(3)) * v_k;
      wrapToPi(x(3) + dt * om_k);
   ];

   % 2. State Jacobian (F) - ∂fx/∂x,  ∂fx/∂y,  ∂fx/∂th;
   %                         ∂fy/∂x,  ∂fy/∂y,  ∂fy/∂th;
   %                         ∂fth/∂x, ∂fth/∂y, ∂fth/∂th;
   F = [
      1, 0, -dt * sin(x(3)) * v_k;
```

```
   0, 1, dt * cos(x(3)) * v_k;
   0, 0, 1;
];

% 3. Process Noise Jacobian (L) - ∂fx/∂wv,  ∂fx/∂ww;
%                                 ∂fy/∂wv,  ∂fy/∂ww;
%                                 ∂fth/∂wv, ∂fth/∂ww;
L = [
   dt * cos(x(3)), 0;
   dt * sin(x(3)), 0;
   0, dt
];

% 4. Prediction Error Covariance Matrix - Prior P (P_prior)
P_pri = F * P * F' + L * Q * L';

% 5. Number of landmark from [my_measurement.mat]
% row_l = size(l, 1);
% row_l = 1;

% 6. Update the state for further calculation in landmark loop
x = x_pri;
P = P_pri;

for j = 1:LM_row
% for j = 1:2

   % 7. Get the position of each landmark in 2D (x, y)
   x_l = l(j, 1);
   y_l = l(j, 2);
```

```
% 8. Predict range measurement
% -  formula: √((x_l − x_k − d cos θ_k)^2 + (y_l − y_k − d sin θ_k)^2)
dx = x_l - x(1) - d * cos(x(3));
dy = y_l - x(2) - d * sin(x(3));

r_pred = sqrt(dx^2 + dy^2);

% Sanity Check
% - Protect the calculation in 12. innovation Jacobian(H) since r_pred as denominator
if r_pred < 1e-6
   continue;
end

% 9. Predict bearing measurement
% -  formula: atan2(y_l − y_k − d sin θ_k , x_l − x_k − d cos θ_k) − θ_k
b_pred = wrapToPi(atan2(dy, dx) - x(3));

% 10. Get real measurement from lidar data [my_measurement.mat]
r_real = r(k, j);
b_real = b(k, j);

% Sanity Check
% - Exclude nan value of range and bearing measurement in dataset
if isnan(r_real) || isnan(b_real)
   continue;
end

% 11. Calculate innovation
% -   formula：  v_k^l = z_k^l(real measurement) − ẑ_k^l(predict measurement)
r_inno = r_real - r_pred;
b_inno = wrapToPi(b_real - b_pred);
```

```
innovation = [r_inno; b_inno];


r_inno_hist(k, j) = r_inno;
b_inno_hist(k, j) = b_inno;


% 12. Innovation Jacobian (H) - ∂r/∂x, ∂r/∂y, ∂r/∂th;
%                              ∂b/∂x, ∂b/∂y, ∂b/∂th;
H = [
   -dx/r_pred, -dy/r_pred, d/r_pred*(dx*sin(x(3)) - dy*cos(x(3)));
   dy/r_pred^2, -dx/r_pred^2, -d/r_pred^2*(dy*sin(x(3)) + dx*cos(x(3)))-1;
];


% 13. Innovation Covariance Matrix
% -   formula: S = HPH' + R
S = H * P * H' + R;


% 14. Kalman Gain
K = P * H' / S;


% 15. Update State - post state (x)
x = x + K * innovation;
x(3) = wrapToPi(x(3));


% 16 Update Prediction Error Covariance Matrix - post error matrix (P)
% Joseph form: P = ( I − K H ) P ( I − K H ) T + K R K T
% Simplified form:P = ( I − K H ) P
% P = (eye(3) - K * H) * P;
I = eye(3);
P = (I - K * H) * P * (I - K * H)' + K * R * K';


% 17. NIS (Normalized Innovation Squared)
```

```
    % formula NIS = z(innovation)'/S(innovation covariance)*z(innovation)
    NIS = innovation' / S * innovation;
    NIS_hist(k, j) = NIS;

  end

  % Update Print Information of state (x)
  x_hist(:,k+1) = x;
  % Update Print Information of error (P)
  P_hist(:,:,k+1) = P;

end

% draw animation of car moving
animate_ekf(x_hist, P_hist, l, t);

% Consistency Evaluation by innovation of range and bearing measurement
analysis_innovation(r_inno_hist, b_inno_hist, t);

% Consistency Evaluation by NIS (Normalized Innovation Squared)

analysis_NIS(NIS_hist, mea_dimension, chi_confi_range);

% Stability Evaluation by Prediction Error Covariance Matrix (P)
analysis_P(P_hist, t);
```

## animate_ekf.m

```
function animate_ekf(x_hist, P_hist, l, t)
  %% ============================================================
  %  CANVAS INITIALISATION
  % ============================================================
```

```
% Open a empty canvas for drawing
figure; clf;

% Show grid
grid on;

% mulity layers
hold on;

% x-axis equals y-axis
axis equal;

% add label for x and y-axis
xlabel('x [m]');
ylabel('y [m]');

N_u = length(t) - 1;

%% ============================================================
%  ANIMATION PROCESS
% ============================================================

% Draw all landmarks
plot(l(:,1), l(:,2), 'kx', 'LineWidth', 1.5, 'MarkerSize', 8);

% Initialized a trajectory object
traj = plot(nan, nan, '-', 'LineWidth', 1.5);

% Initialized a car object
car = plot(nan, nan, 'bo', 'MarkerSize', 8, 'LineWidth', 2);
```

```
% Initialized a heading object
head = quiver(0, 0, 0, 0, 0, 'b', 'LineWidth', 2);

ell = plot(nan, nan, 'r-', 'LineWidth', 1.2);   % ellipse handle

% Set a heading length
Lh = 0.3;

% Animation update process
for k = 1:N_u

   % Get specific state in each timestep
   xk = x_hist(1,k);
   yk = x_hist(2,k);
   th = x_hist(3,k);

   % update trajectory
   set(traj, 'XData', x_hist(1,1:k), 'YData', x_hist(2,1:k));
   % update car position
   set(car, 'XData', xk, 'YData', yk);
   % update car heading
   set(head, 'XData', xk, 'YData', yk, 'UData', Lh*cos(th), 'VData', Lh*sin(th));

   mu = x_hist(1:2,k);
   Pxy = P_hist(1:2,1:2,k);
   vis_scale = 100;
   XY = cov_ellipse_points(mu, Pxy, 2*vis_scale);
   set(ell, 'XData', XY(1,:), 'YData', XY(2,:));

   drawnow;
```

```
        pause(0.01);

    end
end

function XY = cov_ellipse_points(mu_xy, P_xy, n_sigma)

    if nargin < 3 || isempty(n_sigma), n_sigma = 2; end

    P_xy = 0.5 * (P_xy + P_xy');
    [V, D] = eig(P_xy);
    [d, idx] = sort(diag(D), 'descend');
    V = V(:, idx);
    d = max(d, 0);

    a = n_sigma * sqrt(d(1));
    b = n_sigma * sqrt(d(2));

    tt = linspace(0, 2*pi, 200);
    E = [a*cos(tt); b*sin(tt)];
    XY = V * E + mu_xy(:);
end
```

## analysis_innovation.m

```
function analysis_innovation(r_inno_hist, b_inno_hist, t)

    % Draw a range innovation cruve
    figure; grid on; hold on;
```

```
plot(t(1:end-1), r_inno_hist);
title('Range Innovation');
xlabel('t [s]');
ylabel('range innovation [m]');

% Print Mean of range innovation
% - E[z~k]≈0 - unbiasedness
all_r = r_inno_hist(:);
all_r = all_r(~isnan(all_r));
mu_r = mean(all_r);
fprintf('Mean range innovation: %.4f m\n', mu_r);

% Print Standard deviation of range innovation
% - Variance Matching - compare the std_r with the measurement noise (R) range setting
std_r = std(all_r);
fprintf('Standard Deviation of range innovation: %.4f m\n', std_r);

% Draw a bearing innovation cruve
figure; grid on; hold on;
plot(t(1:end-1), b_inno_hist);
title('Bearing Innovation');
xlabel('t [s]');
ylabel('Bearing innovation [rad]');

% Calculate and print mean of bearing innovation
% - E[z~k]≈0 - unbiasedness
all_b = b_inno_hist(:);
all_b = all_b(~isnan(all_b));
mu_b = mean(all_b);
fprintf('Mean bearing innovation: %.4f rad\n', mu_b);
```

```
    % Print Standard deviation of range innovation
    % - Variance Matching - compare the std_b with the measurement noise (R) bearing setting
    std_b = std(all_b);
    fprintf('Standard Deviation of bearing innovation: %.4f rad\n', std_b);

end
```

## analysis_NIS.m

```
function analysis_NIS(NIS_hist, m, c_range)

    % Flatten nis data
    all_nis = NIS_hist(:);
    all_nis = all_nis(~isnan(all_nis));

    figure; hold on; grid on;

    % draw all data as .
    plot(all_nis, '.');
    % draw the range limitation
    upper = chi2inv(c_range, m);
    lower = chi2inv((1-c_range), m);
    yline(upper, 'r--');
    yline(lower, 'r--');

    title('NIS Consistency Check');
    xlabel('measurement index');
    ylabel('NIS');

end
```

## analysis_P.m

```
function analysis_P(P_hist, t)

    % Get variance x and sqrt
    sig_x = sqrt(squeeze(P_hist(1,1,:)));
    % Get variance y and sqrt
    sig_y = sqrt(squeeze(P_hist(2,2,:)));
    % Get variance th and sqrt
    sig_th = sqrt(squeeze(P_hist(3,3,:)));

    figure; grid on; hold on;

    plot(t, sig_x, 'LineWidth', 1.5);
    plot(t, sig_y, 'LineWidth', 1.5);
    plot(t, sig_th, 'LineWidth', 1.5);

    xlabel('t[s]');
    ylabel('Standard Deviation (1-\sigma)');
    legend('\sigma_x [m]',  '\sigma_y [m]', '\sigma_\theta [rad]');
    title('State Uncertainty Evolution');

end
```

## 02_Programming of Task 2 – Dubins path and carrot chasing Algorithm

### main.m

```
clc; clear; close all;

%% ============================================================
%  Variable Initialisation
% ============================================================

% Waypoint list 6*3 matrix
waypoint = [
    [0,   10,  deg2rad(0)];
    [60,  60,  deg2rad(45)];
    [80,  120, deg2rad(30)];
    [150, 70,  deg2rad(-90)];
    [100, 30,  deg2rad(-120)];
    [50,  10,  deg2rad(-180)];
]; % x_position in [m], y_position in [m], heading in [rad]
% Number of waypoint
n_wayp = size(waypoint, 1);
% return radius
r_radius = 5; %[m]
% Circle centre list to save turn lef or right circle centre position
cc_list = nan(n_wayp, 4);
% Distance list
D_list = nan(n_wayp - 1, 22);

%% ============================================================
%  Canvas Initialisation
% ============================================================
figure; grid on; hold on;
axis equal;
```

```
% Draw all waypoint initial pose and heading
for i = 1:n_wayp
    draw_waypoints(waypoint(i,:));
end

%% ================================================================
%  Dubins Path Calculation
% ================================================================

% Left and Right Circle center calculation
for i = 1:n_wayp
    % Depending on waypoint state and return radius to compute
    [l_x_c, l_y_c, r_x_c, r_y_c] =  path_planning.calc_circle_centre(waypoint(i,:), r_radius);
    cc_list(i,:) =  [l_x_c, l_y_c, r_x_c, r_y_c];
    % Draw left with blue color
    draw_circle(l_x_c, l_y_c, r_radius, 'r--');
    % Draw right circle with green color
    draw_circle(r_x_c, r_y_c, r_radius, 'g--');
end

% Distance Calculation in LSL, RSR, LSR, RSL
% for i = 2:2
for i = 1:n_wayp - 1

    % D_list(i,:) = path_planning.dubins_path_planning(waypoint(i,:), waypoint(i+1,:), cc_list(i,:),
cc_list(i+1,:), r_radius);
    % Initialised candidate list
    candi_list = nan(4, 9);
    % LSL
     [d, t_s, the_0_s_t, t_f, the_0_f_t, the_0_s_s, the_0_f_f] = path_planning.CSC('L','L',waypoint(i,:),
waypoint(i+1,:), cc_list(i,:),cc_list(i+1,:), r_radius);
    candi_list(1,:) = [d, t_s, the_0_s_t, t_f, the_0_f_t, the_0_s_s, the_0_f_f];
    % RSR
```

```
    [d, t_s, the_0_s_t, t_f, the_0_f_t, the_0_s_s, the_0_f_f] = path_planning.CSC('R','R',waypoint(i,:), waypoint(i+1,:), cc_list(i,:),cc_list(i+1,:), r_radius);
    candi_list(2,:) =  [d, t_s, the_0_s_t, t_f, the_0_f_t, the_0_s_s, the_0_f_f];
    % LSR
    [d, t_s, the_0_s_t, t_f, the_0_f_t, the_0_s_s, the_0_f_f] = path_planning.CSC('L','R',waypoint(i,:), waypoint(i+1,:), cc_list(i,:),cc_list(i+1,:), r_radius);
    candi_list(3,:) =  [d, t_s, the_0_s_t, t_f, the_0_f_t, the_0_s_s, the_0_f_f];
    % RSL
    [d, t_s, the_0_s_t, t_f, the_0_f_t, the_0_s_s, the_0_f_f] = path_planning.CSC('R','L',waypoint(i,:), waypoint(i+1,:), cc_list(i,:),cc_list(i+1,:), r_radius);
    candi_list(4,:) =  [d, t_s, the_0_s_t, t_f, the_0_f_t, the_0_s_s, the_0_f_f];

    % Gain the minimun value
    vals = candi_list(:,1);
    vals(~isfinite(vals)) = nan;
    [min_val, idx] = min(vals, [], "omitnan");

    % Sort of Dataset
    switch(idx)
        case 1
            C1 = 76; %'L'
            C2 = 76; %'L'
            cc_s = cc_list(i,1:2);
            cc_f = cc_list(i+1,1:2);
        case 2
            C1 = 82; %'R'
            C2 = 82; %'R'
            cc_s = cc_list(i,3:4);
            cc_f = cc_list(i+1,3:4);
        case 3
            C1 = 76; %'L'
            C2 = 82; %'R'
            cc_s = cc_list(i,1:2);
            cc_f = cc_list(i+1,3:4);
        case 4
```

```
        C1 = 82; %'R'
        C2 = 76; %'L'
        cc_s = cc_list(i,3:4);
        cc_f = cc_list(i+1,1:2);
    end

    D_list(i,:) = [idx, waypoint(i,:), waypoint(i+1,:), cc_s, cc_f, candi_list(idx,:), C1, C2];
    % D_list 1:idx, 2-4: x_s,y_s,theta_s; 5-7: x_f, y_f, theta_f; 8-9:cc_s_x, cc_s_y;
    %       10-11: cc_f_x, cc_f_y; 12: path distance; 13-14:Tangent Point S x, y; 15: Tangent angle the_0_s_t
    %       16-17: Tangent Point F x,y; 18: Tangent angle the_0_f_t ; 21-22:  C1(L or R),  C2(L or R)
    path_planning.draw_dubins_path(D_list(i,:), r_radius, 'b-');
end
%% ================================================================
%  Carrot Chasing
% ================================================================

path_following.carrot_chasing(D_list, r_radius);

%% ================================================================
%  Draw Left and Right Circle
% ================================================================
function draw_circle(cx, cy, r, style)

    % Generate 200 points from 0 to 2pi
    ang = linspace(0, 2*pi, 200);

    x = cx + r * cos(ang);
    y = cy + r * sin(ang);

    plot(x, y, style, 'LineWidth', 1);

end

%% ================================================================
```

```
%  Draw Waypoint Position and Heading
% ================================================================
function draw_waypoints(waypoint)

    % Define the length of heading arrow
    L = 5;

    x = waypoint(1);
    y = waypoint(2);
    th= waypoint(3);
    % Draw the specific position
    plot(x, y, 'ko', 'MarkerSize', 8, 'LineWidth', 2);
    % Draw the heading
    quiver(x, y, L*cos(th), L*sin(th), 0, 'r', 'LineWidth', 2);

end
```

## +path_planning / dubins_path_planning.m

```
function D_list = dubins_path_planning(waypoint_s, waypoint_f, cc_list_s, cc_list_f, r_radius)

    % Initialised candidate list

    candi_list = nan(4, 9);

    % LSL

     [d, t_s, the_0_s_t, t_f, the_0_f_t, the_0_s_s, the_0_f_f] = path_planning.CSC('L','L',waypoint_s,
waypoint_f, cc_list_s, cc_list_f, r_radius);

    candi_list(1,:) = [d, t_s, the_0_s_t, t_f, the_0_f_t, the_0_s_s, the_0_f_f];

    % RSR

     [d, t_s, the_0_s_t, t_f, the_0_f_t, the_0_s_s, the_0_f_f] = path_planning.CSC('R','R',waypoint_s,
waypoint_f, cc_list_s, cc_list_f, r_radius);

    candi_list(2,:) =  [d, t_s, the_0_s_t, t_f, the_0_f_t, the_0_s_s, the_0_f_f];

    % LSR

     [d, t_s, the_0_s_t, t_f, the_0_f_t, the_0_s_s, the_0_f_f] = path_planning.CSC('L','R',waypoint_s,
waypoint_f, cc_list_s, cc_list_f, r_radius);

    candi_list(3,:) =  [d, t_s, the_0_s_t, t_f, the_0_f_t, the_0_s_s, the_0_f_f];

    % RSL

     [d, t_s, the_0_s_t, t_f, the_0_f_t, the_0_s_s, the_0_f_f] = path_planning.CSC('R','L',waypoint_s,
waypoint_f, cc_list_s, cc_list_f, r_radius);

    candi_list(4,:) =  [d, t_s, the_0_s_t, t_f, the_0_f_t, the_0_s_s, the_0_f_f];


    % Gain the minimun value

    vals = candi_list(:,1);

    vals(~isfinite(vals)) = nan;

    [min_val, idx] = min(vals, [], "omitnan");


    % Sort of Dataset

    switch(idx)

        case 1

            C1 = 76; %'L'

            C2 = 76; %'L'

            cc_s = cc_list_s(1:2);

            cc_f = cc_list_f(1:2);

        case 2

            C1 = 82; %'R'
```

```
            C2 = 82; %'R'
            cc_s = cc_list_s(3:4);
            cc_f = cc_list_f(3:4);
        case 3
            C1 = 76; %'L'
            C2 = 82; %'R'
            cc_s = cc_list_s(1:2);
            cc_f = cc_list_f(3:4);
        case 4
            C1 = 82; %'R'
            C2 = 76; %'L'
            cc_s = cc_list_s(3:4);
            cc_f = cc_list_f(1:2);
    end

    D_list = [idx, waypoint_s, waypoint_f, cc_s, cc_f, candi_list(idx,:), C1, C2];
end
```

## +path_planning / CSC.m

```
%% ================================================================
%  CSC Distance Calculation
% ================================================================
function [d, t_s, the_0_s_t, t_f, the_0_f_t, the_0_s_s, the_0_f_f] = CSC(C1, C2, waypoint1, waypoint2, cc1, cc2, r)

    % Start Circle - Initial waypoint state
    w_x_s = waypoint1(1,1);
    w_y_s = waypoint1(1,2);
    w_th_s = waypoint1(1,3);

    % End Circle - Initial waypoint state
    w_x_f = waypoint2(1,1);
    w_y_f = waypoint2(1,2);
    w_th_f = waypoint2(1,3);

    % Start Circle - Initial circle centre
    if C1 == 'L'
        c_x_s = cc1(1);
        c_y_s = cc1(2);
    elseif C1 == 'R'
        c_x_s = cc1(3);
        c_y_s = cc1(4);
    else
        error("CSC C1 must be 'L' or 'R'");
    end
    % End Circle - Initial circle centre
    if C2 == 'L'
        c_x_f = cc2(1);
        c_y_f = cc2(2);
    elseif C2 == 'R'
        c_x_f = cc2(3);
```

```
    c_y_f = cc2(4);
else
    error("CSC C2 must be 'L' or 'R'");
end

% Center-to-center distance calculation
dx = c_x_f - c_x_s;
dy = c_y_f - c_y_s;
d_cc = norm([dx, dy]);
% center-to-center angle
the_cc = atan2(dy, dx);
% OUTER tangent
if C1 == C2 % LSL, RSR
    if C1 == 'L'
        the_tan = the_cc - pi/2;
        turn_sign_s = +1;
        turn_sign_f = +1;
    else
        the_tan = the_cc + pi/2;
        turn_sign_s = -1;
        turn_sign_f = -1;
    end
    % tangent point of start circle
    t_x_s = c_x_s + r * cos(the_tan);
    t_y_s = c_y_s + r * sin(the_tan);
    t_s = [t_x_s, t_y_s];

    % tangent point of end circle
    t_x_f = c_x_f + r * cos(the_tan);
    t_y_f = c_y_f + r * sin(the_tan);
    t_f = [t_x_f, t_y_f];
    % distance of the external tangent
    d_t = norm([t_x_f - t_x_s, t_y_f - t_y_s]);
    % Sanity Check - in LSL case d_cc should equal d_t
```

```
    if abs(d_cc - d_t) > 1e-6
        warning("CSC outer: d_cc != d_t (%.6f)", abs(d_cc - d_t));
    end

    % start circle - arc length calculation
    the_0_s_s = atan2(w_y_s - c_y_s, w_x_s - c_x_s);
    the_0_s_t = atan2(t_y_s - c_y_s, t_x_s - c_x_s);
    dthe_s_s_t = wrapTo2Pi(turn_sign_s * (the_0_s_t - the_0_s_s));

    L_arc_s = r * dthe_s_s_t;

    % end circle - arc length calculation
    the_0_f_f = atan2(w_y_f - c_y_f, w_x_f - c_x_f);
    the_0_f_t = atan2(t_y_f - c_y_f, t_x_f - c_x_f);
    dthe_f_t_f = wrapTo2Pi(turn_sign_f * (the_0_f_f - the_0_f_t));

    L_arc_f = r * dthe_f_t_f;

    % LSL length d
    d = L_arc_s + d_t + L_arc_f;
% INNER tangent
else
    % No answer in this case
    if d_cc < 2*r
        d = inf; t_s = [nan nan]; t_f = [nan nan];
        return;
    end

    alpha = acos(2*r / d_cc);

    if C1 == 'L' && C2 == 'R'
        theta = the_cc - alpha;
    else
        theta = the_cc + alpha;
```

```
end

if C1=='L' && C2 =='R'
    the_tan_s = theta;
    the_tan_f = theta + pi;
    turn_sign_s = +1;
    turn_sign_f = -1;
elseif C1=='R' && C2 =='L'
    the_tan_s = theta;
    the_tan_f = theta + pi;
    turn_sign_s = -1;
    turn_sign_f = +1;
end

% tangent point of start circle
t_x_s = c_x_s + r * cos(the_tan_s);
t_y_s = c_y_s + r * sin(the_tan_s);
t_s = [t_x_s, t_y_s];

% tangent point of end circle
t_x_f = c_x_f + r * cos(the_tan_f);
t_y_f = c_y_f + r * sin(the_tan_f);
t_f = [t_x_f, t_y_f];

% Sanity Check - tangent line perpendicular at r
v_line = [t_x_f - t_x_s, t_y_f - t_y_s];
v_rad_s = [t_x_s - c_x_s, t_y_s - c_y_s];
if abs(dot(v_line, v_rad_s)) > 1e-6, warning('INNER tangent not perpendicular at start'); end

% distance of the external tangent
d_t = norm([t_x_f - t_x_s, t_y_f - t_y_s]);

% start circle - arc length calculation
the_0_s_s = atan2(w_y_s - c_y_s, w_x_s - c_x_s);
```

```
        the_0_s_t = atan2(t_y_s - c_y_s, t_x_s - c_x_s);
        dthe_s_s_t = wrapTo2Pi(turn_sign_s * (the_0_s_t - the_0_s_s));

        L_arc_s = r * dthe_s_s_t;

        % end circle - arc length calculation
        the_0_f_f = atan2(w_y_f - c_y_f, w_x_f - c_x_f);
        the_0_f_t = atan2(t_y_f - c_y_f, t_x_f - c_x_f);
        dthe_f_t_f = wrapTo2Pi(turn_sign_f * (the_0_f_f - the_0_f_t));

        L_arc_f = r * dthe_f_t_f;

        % LSL length d
        d = L_arc_s + d_t + L_arc_f;

    end
end
```

## +path_planning / calc_circle_centre.m

```
%% ================================================================
%  Turn Left or Right Circle Centre Calculation
% ================================================================
function [l_x_c, l_y_c, r_x_c, r_y_c] = calc_circle_centre(waypoint, r)

    x = waypoint(1);
    y = waypoint(2);
    th= waypoint(3);

    l_x_c = x + r * cos(th + pi/2);
    l_y_c = y + r * sin(th + pi/2);

    r_x_c = x + r * cos(th - pi/2);
    r_y_c = y + r * sin(th - pi/2);

end
```

## +path_planning / draw_dubins_path.m

```
%% ================================================================
%  Draw A Arc
% ================================================================
function draw_dubins_path(D, r, style)

    [idx, ...
    x_s, y_s, th_s, ...
    x_f, y_f, th_f, ...
    x_s_c, y_s_c, ...
    x_f_c, y_f_c, ...
    d, ...
    x_s_t, y_s_t, the_0_s_t, ...
    x_f_t, y_f_t, the_0_f_t, ...
    the_0_s_s, the_0_f_f, ...
```

```
C1, C2] = data.parse_D_list(D);

% Start Circle - rad path calculation
the_0_s = atan2(y_s - y_s_c, x_s - x_s_c);
the_0_t = atan2(y_s_t - y_s_c, x_s_t - x_s_c);
if char(C1) == 'L'
    dth = wrapTo2Pi(the_0_t - the_0_s);
    % Draw the rad
    theta = linspace(the_0_s, the_0_s + dth, 100);
else % 'R'
    dth = wrapTo2Pi(the_0_s - the_0_t);
    % Draw the rad
    theta = linspace(the_0_s, the_0_s - dth, 100);
end

x = x_s_c + r * cos(theta);
y = y_s_c + r * sin(theta);
plot(x, y, style, 'LineWidth', 2);

% Draw S between tangent points
draw_point(x_s_t, y_s_t, 'ro');
draw_point(x_f_t, y_f_t, 'ro');
plot([x_s_t, x_f_t], [y_s_t, y_f_t], style, "LineWidth", 2);

% End Circle - rad path calculation
the_0_enter = atan2(y_f_t - y_f_c, x_f_t - x_f_c);
the_0_final = atan2(y_f - y_f_c, x_f - x_f_c);

if char(C2) == 'L'
    dth_f = wrapTo2Pi(the_0_final - the_0_enter);
    theta_f = linspace(the_0_enter, the_0_enter + dth_f, 100);
else % 'R'
    dth_f = wrapTo2Pi(the_0_enter - the_0_final);
    theta_f = linspace(the_0_enter, the_0_enter - dth_f, 100);
```

```
    end

    xx = x_f_c + r * cos(theta_f);
    yy = y_f_c + r * sin(theta_f);
    plot(xx, yy, style, 'LineWidth', 2);

end

%% ================================================================
%  Draw A Point
% ================================================================
function draw_point(x, y, style)

    plot(x, y, style, 'MarkerSize', 4, 'LineWidth', 1, 'MarkerFaceColor', 'y');

end
```

## +path_following / carrot_chasing.m

```
function carrot_chasing(D_list, r_radius)
    %% ================================================================
    %  Variable Initialisation
    % ================================================================
    % UGV velocity
    v = 5; %[m/s]
    % Total time
    T = 1000; %[s]
    % Timestep
    dt = 0.05; %[s]
    % Max lateral acceleration
    u_max = 5; %[m/s^2]
    % Count Time
    N_t = T/dt;
    % Car Movement Initialisation
    car_x = 0;
    car_y = 10;
    car_psi = deg2rad(0);
    % Car movement
    car = plot(nan, nan, 'bo', 'MarkerSize', 8);
    arrow = quiver(0,0,0,0,'r');
    % Gain
    kpsi = 2.0;
    % Look ahead meter for line
    delta_lahead = 5; %[m]
    % Look ahead for arc
    lamuda_lahead = 0.4; %[rad]
    % Main Path segment state
    seg = 1;
    % Sub-Path segment state
    path_idx = 1;
    % Distance threshold
```

```
eps = 1; %[m]


% --------  Initial State Parse  --------
[idx, ...
x_s, y_s, th_s, ...
x_f, y_f, th_f, ...
x_s_c, y_s_c, ...
x_f_c, y_f_c, ...
d, ...
x_s_t, y_s_t, the_0_s_t, ...
x_f_t, y_f_t, the_0_f_t, ...
the_0_s_s, the_0_f_f, ...
C1, C2] = data.parse_D_list(D_list(seg,:));


% Draw the first part straight path
plot([x_s_t x_f_t],[y_s_t y_f_t], 'y--', 'LineWidth', 1.5);


%% ================================================================
%  Main Loop (Carror & Chasing)
% ================================================================
for i = 1:N_t


    % Depending on distance between car and start circle tagent point to ensure the path segment idx allocation
    if path_idx == 1 && norm([car_x - x_s_t, car_y - y_s_t]) < eps
        path_idx = 2;
    elseif path_idx == 2 && norm([car_x - x_f_t, car_y - y_f_t]) < eps
        path_idx = 3;
    end


    % Sub-Path segment idx 1: in start circle -> 2: in straight line -> 3: in final circle; the idx can not being backward deliver
    if path_idx == 1
        [carrot, th_carrot, sgn] = chasing_arc( x_s_c, y_s_c, the_0_s_s, the_0_s_t, C1, r_radius, car_x, car_y, lamuda_lahead);
```

```
            % desired heading calculation
            psi_tan = wrapToPi(th_carrot + sgn*pi/2);
            psi_los = atan2(carrot(2)-car_y, carrot(1)-car_x);
            psi_d = wrapToPi(0.7*psi_tan + 0.3*psi_los);
        elseif path_idx == 2
            carrot = chasing_line([x_s_t, y_s_t], [x_f_t, y_f_t], car_x, car_y, delta_lahead, i
);
            psi_d = atan2(carrot(2)-car_y, carrot(1)-car_x);
        else
            [carrot, th_carrot, sgn] = chasing_arc(x_f_c, y_f_c, the_0_f_t, the_0_f_f, C2, r_radius, car_x,
car_y, lamuda_lahead);
            psi_tan = wrapToPi(th_carrot + sgn*pi/2);
            psi_los = atan2(carrot(2)-car_y, carrot(1)-car_x);
            psi_d = wrapToPi(0.7*psi_tan + 0.3*psi_los);
        end

        % heading control
        epsi = wrapToPi(psi_d - car_psi);
        % lateral acceleration - it is not control low , it is rate control by gain
        u = kpsi * epsi * v;
        u = max(min(u, u_max), -u_max);

        % dynamics
        car_x = car_x + v * cos(car_psi) * dt;
        car_y = car_y + v * sin(car_psi) * dt;
        car_psi = car_psi + (u/v) * dt;

        % draw
        set(car, 'XData', car_x, 'YData', car_y);
        set(arrow, 'XData', car_x, 'YData', car_y, 'UData', 2*cos(car_psi), 'VData', 2*sin(car_psi));
        drawnow;

        % Main Path State Check: the car is close to final point, state change to next part
        if norm([car_x - x_f, car_y - y_f]) < eps
            if seg < size(D_list, 1)
```

```
            % Next State Update
            seg = seg + 1;
            path_idx = 1;
            % --------  Update Path Segment Parameters  --------
            [idx, ...
            x_s, y_s, th_s, ...
            x_f, y_f, th_f, ...
            x_s_c, y_s_c, ...
            x_f_c, y_f_c, ...
            d, ...
            x_s_t, y_s_t, the_0_s_t, ...
            x_f_t, y_f_t, the_0_f_t, ...
            the_0_s_s, the_0_f_f, ...
            C1, C2] = data.parse_D_list(D_list(seg,:));
            % draw the straight line as path
            plot([x_s_t x_f_t],[y_s_t y_f_t], 'y--', 'LineWidth', 1.5);
        else
            break;
        end
    end

end

%% ================================================================
%  Get Carrot in ARC
% ================================================================
function [carrot, th_carrot, sgn] = chasing_arc(cx , cy, th_start, th_end, C, r, car_x, car_y, look_ahead)

    % Arc length calculation, sgn showing the turning direction, detal showing the turning length
    if char(C) == 'L'
        arc_length = wrapTo2Pi(th_end - th_start);
        sgn = +1;
```

```
else
    arc_length = wrapTo2Pi(th_start - th_end);
    sgn = -1;
end

% Projection car position to circle
th_car = atan2(car_y - cy, car_x - cx);

% Car progress evaluation: Calculate the distance between start and final
if char(C) == 'L'
    car_progress = wrapTo2Pi(th_car - th_start);
else
    car_progress = wrapTo2Pi(th_start - th_car);
end

% Just in case to prevent progress greater than total distance of arc
car_progress = min(max(car_progress, 0), arc_length);

% carrot length in arc - Δθ=s/r
% dth_la = look_ahead / r;
dth_la = look_ahead;

% carrot theta respect to progress
prog2 = min(car_progress + dth_la, arc_length);

% carrot theta respect to zero
th_carrot = th_start + sgn * prog2;

% carrot position
carrot = [cx + r*cos(th_carrot), cy + r*sin(th_carrot)];



plot(carrot(1), carrot(2), 'o', 'Color', [1 0.5 0], "MarkerSize",5, "LineWidth",5);
```

```
end

%% ================================================================
%  Get Carrot in Line
% ================================================================
function carrot = chasing_line(t_entry, t_exit, car_x, car_y, look_ahead)

    % distance of tangent line
    dtx = t_exit(1) - t_entry(1);
    dty = t_exit(2) - t_entry(2);
    d_t = norm([dtx, dty]);

    % direction of tangent line （unit tangent vector）
    utv = [dtx, dty] / d_t;

    % progress of the car in the tangent path - calculate the projection of car position
    car_progress = dot([car_x, car_y] - t_entry, utv);
    car_progress = min(max(car_progress, 0), d_t);

    % carrot position
    carrot = t_entry + min(car_progress + look_ahead, d_t) * utv;

    % plot(carrot(1), carrot(2), 'ro', "MarkerSize",5, "LineWidth",5);
        if mod(i, 50) == 0
            plot(carrot(1), carrot(2), 'o', 'Color', [1 0.5 0], "MarkerSize",5, "LineWidth",5);
        end


end
```

## +data / parse_D_lsit.m

```
function [idx, ...
 x_s, y_s, th_s, ...
 x_f, y_f, th_f, ...
 x_s_c, y_s_c, ...
 x_f_c, y_f_c, ...
 d, ...
 x_s_t, y_s_t, the_0_s_t, ...
 x_f_t, y_f_t, the_0_f_t, ...
 the_0_s_s, the_0_f_f, ...
 C1, C2] = parse_D_list(D)
% D (1 x 22) layout
% -------------------------------------------------------------------------
%  1  : idx              (path type index, e.g., LSL/RSR/...)
%
%  2  : x_s              (start waypoint x)
%  3  : y_s              (start waypoint y)
%  4  : th_s             (start heading angle)
%
%  5  : x_f              (final waypoint x)
%  6  : y_f              (final waypoint y)
%  7  : th_f             (final heading angle)
%
%  8  : x_s_c            (start circle center x)
%  9  : y_s_c            (start circle center y)
%
% 10  : x_f_c            (final circle center x)
% 11  : y_f_c            (final circle center y)
%
% 12  : d                (total Dubins path distance)
%
% 13  : x_s_t            (start tangent point x)
% 14  : y_s_t            (start tangent point y)
```

```
% 15  : the_0_s_t         (start circle → tangent exit angle)
%
% 16  : x_f_t             (final tangent point x)
% 17  : y_f_t             (final tangent point y)
% 18  : the_0_f_t         (final circle → tangent entry angle)
%
% 19  : the_0_s_s         (start circle → start pose angle)
% 20  : the_0_f_f         (final circle → final pose angle)
%
% 21  : C1                (start turn direction: 'L' or 'R')
% 22  : C2                (final turn direction: 'L' or 'R')
% ------------------------------------------------------------------------

   idx =  D(1);
   x_s = D(2);
   y_s = D(3);
   th_s = D(4);
   x_f = D(5);
   y_f = D(6);
   th_f = D(7);
   x_s_c = D(8);
   y_s_c = D(9);
   x_f_c = D(10);
   y_f_c = D(11);
   d = D(12);
   x_s_t = D(13);
   y_s_t = D(14);
   the_0_s_t = D(15);
   x_f_t = D(16);
   y_f_t = D(17);
   the_0_f_t  = D(18);
   the_0_s_s  = D(19);
   the_0_f_f = D(20);
   C1 = D(21);
```

```
    C2 = D(22);

end
```

## 03_Programming of Task 2 – RRT

### main.m

```
clc; clear; close all;

%% ================================================================
%  Environment Initialisation
% ================================================================

% Initialise ENV
env = common.make_env();

% Initialise Car
car = common.make_car(env.start);

% Canvas Initialisation
common.init_map(env, 1);

%% ================================================================
%  RRT（Rapidly-exploring Random Tree） Path planning
% ================================================================

% Single RRT
[n_added, P_list] = path_planning.rrt.rrt(env);

% Build the Path
order_final_path = path_planning.rrt.rrt_path_build(n_added, P_list, env.goal);

% Canvas Initialisation
common.init_map(env, 2);
```

```
% re-draw the final rrt path on the second figure
plot(order_final_path(:,3), order_final_path(:,4), 'b-', 'LineWidth', 2);

% Optimize to find the short path
short_path = path_planning.rrt.rrt_shortcut_opt(order_final_path, env);

% Canvas Initialisation
common.init_map(env, 3);

% Dubins path planning
D_list = path_planning.dubins.dubins_path(short_path, env);

%% ============================================================
%  Path following
% ============================================================
path_following.carrot_chasing(D_list, car);
```

## +path_planning / rrt / rrt.m

```
function [n_added, P_list] = rrt(env)

%% ============================================================
%  Algorithmic Parameter Settings
% ============================================================
% Node toal number
n_node = 1000;

% Step size
step_size = 15; %[m]

% node idx in tree
n_added = 1;

% Point list
P_list = nan(n_node, 4);
% 1: parent node; 2: current node; 3-4 current position
P_list(n_added,:) = [0, 1, env.start];

% Distance list
d_list = nan(n_node, 1);

%% ============================================================
%  Main Process
% ============================================================
for i = 1:n_node

    % Initialize distance list
    d_list = inf(n_added, 1);
```

```
% Generate random points
x_rand = env.x_min + rand * (env.x_max - env.x_min);
y_rand = env.y_min + rand * (env.y_max - env.y_min);
p_rand = [x_rand, y_rand];

% Draw rand point in each round
plot(x_rand, y_rand, 'ro', 'LineWidth',1, 'MarkerSize',1);
% Showing the round number in title part
title(sprintf("RRT Nodes: %d", n_added));

% Find the nearest point - Calculate the distance between random point and each point
for j = 1:n_added
   % Distance
   d = norm(p_rand - P_list(j,3:4));
   d_list(j) = d;
end

% Get the min distance in the d_list
[m, idx] = min(d_list);
% Get the nearest point
P_nearest = P_list(idx,3:4);
% Direction Vector
d_v = p_rand - P_nearest;
% Distance
d = norm(p_rand - P_nearest);
% Moving direction
th_d = d_v / d;
% Moving distance
p_new = P_nearest + step_size * th_d;
```

```
    % Collision with obstacles - in or on
    result = path_planning.rrt.chk_collision([P_nearest; p_new], env.poly);
    % No collision then update P_list
    if result == 0
        % Save the new point to the list
        P_list(n_added + 1,:) = [idx, n_added + 1, p_new];
        n_added = n_added + 1;
        % Draw the path
        plot([P_list(idx,3:3) p_new(1)], [P_list(idx, 4:4) p_new(2)], 'r-','LineWidth',1.5);
        % Goal Reaching
        d_g = norm(p_new - env.goal);
        if d_g < env.goal_radius
            break;
        end
    end

    drawnow;
end
end
```

## +path_planning / rrt / chk_collision.m

```
function result = chk_collision(line, poly)
% plotting
% chk_collision_plot(line, poly);

result = 0;

% poly should be a cell array, cound how many polygon we need to check
shape_count = size(poly, 2);
```

```
for i = 1:shape_count
   crnt_poly = poly{i};
   poly_size = size(crnt_poly);  % the ith perticular segments

   % polygon should consists of at least 3 points
   if poly_size(1) <= 2
      result = -2;
      return;              % error, incorrect polymer definition
   end

   % check line segement return to initial point.
   % If not, add initial point as ending point to encircle the polygon
   if any(crnt_poly(poly_size(1), :) ~= crnt_poly(1, :))
      poly_size(1) = poly_size(1)+1;
      crnt_poly(poly_size(1), :) = crnt_poly(1, :);
   end
   seg_count = poly_size(1) - 1;   % segment is one count lesser then vertex

   dA = line(2,:) - line(1,:);                          % dimension should be [1, 2]
   dB = crnt_poly(2:poly_size(1), :) - crnt_poly(1:poly_size(1)-1, :);      % dimension should be
[seg_count, 2]
   dA1B1 = repmat(line(1,:), [seg_count, 1]) - crnt_poly(1:seg_count, :);% dimension should be
[seg_count, 2]
   denominator = dB(:, 2) .* dA(1) - dB(:, 1) .*dA(2);
   if all(denominator == 0)       % all lines are parrel, which is very unlikely
      result = 0;
      return;
   end
   ua = dB(:, 1) .* dA1B1(:, 2) - dB(:, 2) .* dA1B1(:, 1);
   ub = dA1B1(:, 2) .* dA(1) - dA1B1(:, 1) .* dA(2);
   ua = ua ./ denominator;
   ub = ub ./ denominator;
```

```
    if ( all( ((ua<0)|(ua>1))|((0>ub)|(ub>1)) ) )
        result = 0;
    else
        result = 1;
        return;
    end
end


end
```

### +path_planning / rrt / rrt_path_build.m

```
function [order_final_path] = rrt_path_build(n_added, P_list, goal)
    final_path = nan(n_added, 4);
    % build the final path
    n_real_node = size(P_list(~isnan(P_list(:,1)),:,1),1);
    clue  = n_added;

    for i = 1:n_real_node

        % Set the final one to path list
        final_path(i,:) = P_list(clue,:);
        % Set new clue
        clue = final_path(i,1:1);

        if clue == 0
            break;
        end

    end
```

```
    % inverse the order of final path as 1 - N
    order_final_path = flipud(final_path);
    % delete nan clomns
    order_final_path = order_final_path(~all(isnan(order_final_path),2),:);
    % add the goal in the end of the path
    n_final_path = size(order_final_path,1);
    order_final_path(n_final_path+1,:) = [n_final_path, n_final_path + 1, goal];

    % draw the final path
    plot(order_final_path(:,3), order_final_path(:,4), 'b-', 'LineWidth', 2);
end
```

## +path_planning / rrt / rrt_shortcut_opt.m

```
function short_path = rrt_shortcut_opt(order_final_path, env)

poly = env.poly;
n = size(order_final_path, 1);
idx = 1;
n_s_node = 1;
short_path = order_final_path(n_s_node,:);

while idx ~= n
    for j = idx:n
        collision = path_planning.rrt.chk_collision([order_final_path(idx,3:4); order_final_path(j,3:4)],
poly);
        if j < n
            if collision == 0
                % idx = idx + 1;
            else
                idx = j - 1;
```

```
            n_s_node = n_s_node + 1;
            short_path(n_s_node,:) = order_final_path(idx,:);
            break;
        end
    else
        if collision == 0
            n_s_node = n_s_node + 1;
            idx = j;
            short_path(n_s_node,:) = order_final_path(idx,:);
            break;
        else
            idx = j - 1;
            n_s_node = n_s_node + 1;
            short_path(n_s_node,:) = order_final_path(idx,:);
            idx = idx + 1;
            n_s_node = n_s_node + 1;
            short_path(n_s_node,:) = order_final_path(idx,:);
            break;
        end

    end

  end

end
% draw the final path
plot(short_path(:,3), short_path(:,4), 'k-', 'LineWidth', 2);
end
```

## +path_planning / dubins / dubins_path.m

```
function D_list = dubins_path(short_path, env)

% Optimize to build Dubins path by short path
n = size(short_path, 1);
waypoint = zeros(n,3);
waypoint(:,1:2) = short_path(:,3:4);
for i = 1:n
  if i == 1
    waypoint(i,3) = deg2rad(0);
  elseif i == n
    waypoint(i,3) = deg2rad(90);
  else
    p_prev = waypoint(i-1, 1:2);
    p_next = waypoint(i+1, 1:2);
    waypoint(i,3) = atan2(p_next(2)-p_prev(2), p_next(1)-p_prev(1));
  end
end

% Number of waypoint
n_wayp = size(waypoint, 1);
% Circle centre list to save turn lef or right circle centre position
cc_list = nan(n_wayp, 4);
% Distance list
D_list = nan(n_wayp - 1, 22);

car = common.make_car(env.start);

% Draw all waypoint initial pose and heading
for i = 1:n_wayp
  draw_waypoints(waypoint(i,:));
```

```
end

% Left and Right Circle center calculation
for i = 1:n_wayp
   % Depending on waypoint state and return radius to compute
   [l_x_c, l_y_c, r_x_c, r_y_c] =  path_planning.dubins.calc_circle_centre(waypoint(i,:), car.r);
   cc_list(i,:) =  [l_x_c, l_y_c, r_x_c, r_y_c];
   % Draw left with blue color
   draw_circle(l_x_c, l_y_c, car.r, 'r--');
   % Draw right circle with green color
   draw_circle(r_x_c, r_y_c, car.r, 'g--');
end

for i = 1:n_wayp - 1

   % D_list(i,:) = path_planning.dubins_path_planning(waypoint(i,:), waypoint(i+1,:), cc_list(i,:),
cc_list(i+1,:), r_radius);
   % Initialised candidate list
   candi_list = nan(4, 9);
   % LSL
    [d, t_s, the_0_s_t, t_f, the_0_f_t, the_0_s_s, the_0_f_f] =
path_planning.dubins.CSC('L','L',waypoint(i,:), waypoint(i+1,:), cc_list(i,:),cc_list(i+1,:), car.r);
   candi_list(1,:) = [d, t_s, the_0_s_t, t_f, the_0_f_t, the_0_s_s, the_0_f_f];
   % RSR
    [d, t_s, the_0_s_t, t_f, the_0_f_t, the_0_s_s, the_0_f_f] =
path_planning.dubins.CSC('R','R',waypoint(i,:), waypoint(i+1,:), cc_list(i,:),cc_list(i+1,:), car.r);
   candi_list(2,:) =  [d, t_s, the_0_s_t, t_f, the_0_f_t, the_0_s_s, the_0_f_f];
   % LSR
    [d, t_s, the_0_s_t, t_f, the_0_f_t, the_0_s_s, the_0_f_f] =
path_planning.dubins.CSC('L','R',waypoint(i,:), waypoint(i+1,:), cc_list(i,:),cc_list(i+1,:), car.r);
   candi_list(3,:) =  [d, t_s, the_0_s_t, t_f, the_0_f_t, the_0_s_s, the_0_f_f];
   % RSL
```

```
    [d, t_s, the_0_s_t, t_f, the_0_f_t, the_0_s_s, the_0_f_f] =
path_planning.dubins.CSC('R','L',waypoint(i,:), waypoint(i+1,:), cc_list(i,:),cc_list(i+1,:), car.r);
    candi_list(4,:) =  [d, t_s, the_0_s_t, t_f, the_0_f_t, the_0_s_s, the_0_f_f];


    % Gain the minimun value
    vals = candi_list(:,1);
    vals(~isfinite(vals)) = nan;
    [min_val, idx] = min(vals, [], "omitnan");


    % Sort of Dataset
    switch(idx)
      case 1
        C1 = 76; %'L'
        C2 = 76; %'L'
        cc_s = cc_list(i,1:2);
        cc_f = cc_list(i+1,1:2);
      case 2
        C1 = 82; %'R'
        C2 = 82; %'R'
        cc_s = cc_list(i,3:4);
        cc_f = cc_list(i+1,3:4);
      case 3
        C1 = 76; %'L'
        C2 = 82; %'R'
        cc_s = cc_list(i,1:2);
        cc_f = cc_list(i+1,3:4);
      case 4
        C1 = 82; %'R'
        C2 = 76; %'L'
        cc_s = cc_list(i,3:4);
        cc_f = cc_list(i+1,1:2);
```

```
    end

    D_list(i,:) = [idx, waypoint(i,:), waypoint(i+1,:), cc_s, cc_f, candi_list(idx,:), C1, C2];
    % D_list 1:idx, 2-4: x_s,y_s,theta_s; 5-7: x_f, y_f, theta_f; 8-9:cc_s_x, cc_s_y;
    %       10-11: cc_f_x, cc_f_y; 12: path distance; 13-14:Tangent Point S x, y; 15: Tangent angle the_0_s_t
    %       16-17: Tangent Point F x,y; 18: Tangent angle the_0_f_t ; 21-22:  C1(L or R),  C2(L or R)
    path_planning.dubins.draw_dubins_path(D_list(i,:), car.r, 'b-');
end

end

%% ================================================================
%  Draw Left and Right Circle
% ================================================================
function draw_circle(cx, cy, r, style)

    % Generate 200 points from 0 to 2pi
    ang = linspace(0, 2*pi, 200);

    x = cx + r * cos(ang);
    y = cy + r * sin(ang);

    plot(x, y, style, 'LineWidth', 1);

end

%% ================================================================
%  Draw Waypoint Position and Heading
% ================================================================
function draw_waypoints(waypoint)
```

```
    % Define the length of heading arrow
    L = 5;

    x = waypoint(1);
    y = waypoint(2);
    th= waypoint(3);
    % Draw the specific position
    plot(x, y, 'ko', 'MarkerSize', 8, 'LineWidth', 2);
    % Draw the heading
    quiver(x, y, L*cos(th), L*sin(th), 0, 'r', 'LineWidth', 2);

end
```

## +path_planning / dubins / calc_circle_centre.m

```
%% ================================================================
%  Turn Left or Right Circle Centre Calculation
% ================================================================
function [l_x_c, l_y_c, r_x_c, r_y_c] = calc_circle_centre(waypoint, r)

    x = waypoint(1);
    y = waypoint(2);
    th= waypoint(3);

    l_x_c = x + r * cos(th + pi/2);
    l_y_c = y + r * sin(th + pi/2);

    r_x_c = x + r * cos(th - pi/2);
    r_y_c = y + r * sin(th - pi/2);
```

```
end
```

## +path_planning / dubins / CSC.m

```
%% ============================================================

%  CSC Distance Calculation

% ============================================================

function [d, t_s, the_0_s_t, t_f, the_0_f_t, the_0_s_s, the_0_f_f] = CSC(C1, C2, waypoint1, waypoint2, cc1, cc2, r)


  % Start Circle - Initial waypoint state

  w_x_s = waypoint1(1,1);

  w_y_s = waypoint1(1,2);

  w_th_s = waypoint1(1,3);


  % End Circle - Initial waypoint state

  w_x_f = waypoint2(1,1);

  w_y_f = waypoint2(1,2);

  w_th_f = waypoint2(1,3);


  % Start Circle - Initial circle centre

  if C1 == 'L'

    c_x_s = cc1(1);

    c_y_s = cc1(2);

  elseif C1 == 'R'

    c_x_s = cc1(3);

    c_y_s = cc1(4);

  else

    error("CSC C1 must be 'L' or 'R'");

  end

  % End Circle - Initial circle centre

  if C2 == 'L'

    c_x_f = cc2(1);
```

```
    c_y_f = cc2(2);
elseif C2 == 'R'
    c_x_f = cc2(3);
    c_y_f = cc2(4);
else
    error("CSC C2 must be 'L' or 'R'");
end

% Center-to-center distance calculation
dx = c_x_f - c_x_s;
dy = c_y_f - c_y_s;
d_cc = norm([dx, dy]);
% center-to-center angle
the_cc = atan2(dy, dx);
% OUTER tangent
if C1 == C2 % LSL, RSR
    if C1 == 'L'
        the_tan = the_cc - pi/2;
        turn_sign_s = +1;
        turn_sign_f = +1;
    else
        the_tan = the_cc + pi/2;
        turn_sign_s = -1;
        turn_sign_f = -1;
    end
    % tangent point of start circle
    t_x_s = c_x_s + r * cos(the_tan);
    t_y_s = c_y_s + r * sin(the_tan);
    t_s = [t_x_s, t_y_s];

    % tangent point of end circle
```

```
    t_x_f = c_x_f + r * cos(the_tan);
    t_y_f = c_y_f + r * sin(the_tan);
    t_f = [t_x_f, t_y_f];
    % distance of the external tangent
    d_t = norm([t_x_f - t_x_s, t_y_f - t_y_s]);
    % Sanity Check - in LSL case d_cc should equal d_t
    if abs(d_cc - d_t) > 1e-6
       warning("CSC outer: d_cc != d_t (%.6f)", abs(d_cc - d_t));
    end

    % start circle - arc length calculation
    the_0_s_s = atan2(w_y_s - c_y_s, w_x_s - c_x_s);
    the_0_s_t = atan2(t_y_s - c_y_s, t_x_s - c_x_s);
    dthe_s_s_t = wrapTo2Pi(turn_sign_s * (the_0_s_t - the_0_s_s));

    L_arc_s = r * dthe_s_s_t;

    % end circle - arc length calculation
    the_0_f_f = atan2(w_y_f - c_y_f, w_x_f - c_x_f);
    the_0_f_t = atan2(t_y_f - c_y_f, t_x_f - c_x_f);
    dthe_f_t_f = wrapTo2Pi(turn_sign_f * (the_0_f_f - the_0_f_t));

    L_arc_f = r * dthe_f_t_f;

    % LSL length d
    d = L_arc_s + d_t + L_arc_f;
% INNER tangent
else
    % No answer in this case
    if d_cc < 2*r
       d = inf; t_s = [nan nan]; t_f = [nan nan];
```

```
    return;
end

alpha = acos(2*r / d_cc);

if C1 == 'L' && C2 == 'R'
    theta = the_cc - alpha;
else
    theta = the_cc + alpha;
end

if C1=='L' && C2 =='R'
    the_tan_s = theta;
    the_tan_f = theta + pi;
    turn_sign_s = +1;
    turn_sign_f = -1;
elseif C1=='R' && C2 =='L'
    the_tan_s = theta;
    the_tan_f = theta + pi;
    turn_sign_s = -1;
    turn_sign_f = +1;
end

% tangent point of start circle
t_x_s = c_x_s + r * cos(the_tan_s);
t_y_s = c_y_s + r * sin(the_tan_s);
t_s = [t_x_s, t_y_s];

% tangent point of end circle
t_x_f = c_x_f + r * cos(the_tan_f);
t_y_f = c_y_f + r * sin(the_tan_f);
```

```
        t_f = [t_x_f, t_y_f];

        % Sanity Check - tangent line perpendicular at r
        v_line = [t_x_f - t_x_s, t_y_f - t_y_s];
        v_rad_s = [t_x_s - c_x_s, t_y_s - c_y_s];
        if abs(dot(v_line, v_rad_s)) > 1e-6, warning('INNER tangent not perpendicular at start'); end

        % distance of the external tangent
        d_t = norm([t_x_f - t_x_s, t_y_f - t_y_s]);

        % start circle - arc length calculation
        the_0_s_s = atan2(w_y_s - c_y_s, w_x_s - c_x_s);
        the_0_s_t = atan2(t_y_s - c_y_s, t_x_s - c_x_s);
        dthe_s_s_t = wrapTo2Pi(turn_sign_s * (the_0_s_t - the_0_s_s));

        L_arc_s = r * dthe_s_s_t;

        % end circle - arc length calculation
        the_0_f_f = atan2(w_y_f - c_y_f, w_x_f - c_x_f);
        the_0_f_t = atan2(t_y_f - c_y_f, t_x_f - c_x_f);
        dthe_f_t_f = wrapTo2Pi(turn_sign_f * (the_0_f_f - the_0_f_t));

        L_arc_f = r * dthe_f_t_f;

        % LSL length d
        d = L_arc_s + d_t + L_arc_f;

    end
end
```

+path_planning / dubins / draw_dubins_path.m

```
%% ================================================================
%  Draw A Arc
% ================================================================
function draw_dubins_path(D, r, style)

  [idx, ...
  x_s, y_s, th_s, ...
  x_f, y_f, th_f, ...
  x_s_c, y_s_c, ...
  x_f_c, y_f_c, ...
  d, ...
  x_s_t, y_s_t, the_0_s_t, ...
  x_f_t, y_f_t, the_0_f_t, ...
  the_0_s_s, the_0_f_f, ...
  C1, C2] = data.parse_D_list(D);

  % Start Circle - rad path calculation
  the_0_s = atan2(y_s - y_s_c, x_s - x_s_c);
  the_0_t = atan2(y_s_t - y_s_c, x_s_t - x_s_c);
  if char(C1) == 'L'
    dth = wrapTo2Pi(the_0_t - the_0_s);
    % Draw the rad
    theta = linspace(the_0_s, the_0_s + dth, 100);
  else % 'R'
    dth = wrapTo2Pi(the_0_s - the_0_t);
    % Draw the rad
    theta = linspace(the_0_s, the_0_s - dth, 100);
  end

  x = x_s_c + r * cos(theta);
```

```
    y = y_s_c + r * sin(theta);
    plot(x, y, style, 'LineWidth', 2);

    % Draw S between tangent points
    draw_point(x_s_t, y_s_t, 'ro');
    draw_point(x_f_t, y_f_t, 'ro');
    plot([x_s_t, x_f_t], [y_s_t, y_f_t], style, "LineWidth", 2);

    % End Circle - rad path calculation
    the_0_enter = atan2(y_f_t - y_f_c, x_f_t - x_f_c);
    the_0_final = atan2(y_f - y_f_c, x_f - x_f_c);

    if char(C2) == 'L'
        dth_f = wrapTo2Pi(the_0_final - the_0_enter);
        theta_f = linspace(the_0_enter, the_0_enter + dth_f, 100);
    else % 'R'
        dth_f = wrapTo2Pi(the_0_enter - the_0_final);
        theta_f = linspace(the_0_enter, the_0_enter - dth_f, 100);
    end

    xx = x_f_c + r * cos(theta_f);
    yy = y_f_c + r * sin(theta_f);
    plot(xx, yy, style, 'LineWidth', 2);

end

%% ============================================================
%  Draw A Point
% ============================================================
function draw_point(x, y, style)
```

```
    plot(x, y, style, 'MarkerSize', 4, 'LineWidth', 1, 'MarkerFaceColor', 'y');

end
```

## +path_following / carrot_chasing.m

```
function carrot_chasing(D_list, car)
    %% ================================================================
    %  Variable Initialisation
    % ================================================================

    r_radius = car.r;
    % UGV velocity
    v = 5; %[m/s]
    % Total time
    T = 1000; %[s]
    % Timestep
    dt = 0.05; %[s]
    % Max lateral acceleration
    u_max = 5; %[m/s^2]
    % Max augular velocity
    ang_v_max = u_max / v;
    % Count Time
    N_t = T/dt;
    % Car Movement Initialisation
    car_x = -30;
    car_y = 30;
    car_psi = deg2rad(180);
    % Car movement
```

```
    car_p = plot(nan, nan, 'bo', 'MarkerSize', 12, 'MarkerFaceColor', [0 0.6 1], 'MarkerEdgeColor', 'k',
'LineWidth', 3);
    arrow = quiver(0, 0, 0, 0,'r', 'LineWidth',2);
    % Car trajectory
    traj = plot(nan, nan, 'r-', 'LineWidth',2);
    x_hist = nan(1, N_t);
    y_hist = nan(1, N_t);
    % Gain
    kpsi = 5.0;
    % Look ahead meter for line
    delta_lahead = 5; %[m]
    % Look ahead for arc
    lamuda_lahead = 0.5; %[rad]
    % Main Path segment state
    seg = 1;
    % Sub-Path segment state
    path_idx = 2;
    % Distance threshold
    eps = 1; %[m]

    % --------  Initial State Parse  --------
    [idx, ...
    x_s, y_s, th_s, ...
    x_f, y_f, th_f, ...
    x_s_c, y_s_c, ...
    x_f_c, y_f_c, ...
    d, ...
    x_s_t, y_s_t, the_0_s_t, ...
    x_f_t, y_f_t, the_0_f_t, ...
    the_0_s_s, the_0_f_f, ...
    C1, C2] = data.parse_D_list(D_list(seg,:));
```

```
% Draw the first part straight path
plot([x_s_t x_f_t],[y_s_t y_f_t], 'y--', 'LineWidth', 1.5);


%% ============================================================
%  Main Loop (Carror & Chasing)
% ============================================================
for i = 1:N_t


    % save trajectory history
    x_hist(i) = car_x;
    y_hist(i) = car_y;


    % Depending on distance between car and start circle tagent point to ensure the path segment idx allocation
    if path_idx == 1 && norm([car_x - x_s_t, car_y - y_s_t]) < eps
        path_idx = 2;
    elseif path_idx == 2 && norm([car_x - x_f_t, car_y - y_f_t]) < eps
        path_idx = 3;
    end


    % Sub-Path segment idx 1: in start circle -> 2: in straight line -> 3: in final circle; the idx can not being backward deliver
    if path_idx == 1
        [carrot, th_carrot, sgn] = chasing_arc( x_s_c, y_s_c, the_0_s_s, the_0_s_t, C1, r_radius, car_x, car_y, lamuda_lahead);
        % desired heading calculation
        psi_tan = wrapToPi(th_carrot + sgn*pi/2);
        psi_los = atan2(carrot(2)-car_y, carrot(1)-car_x);
        psi_d = wrapToPi(0.7*psi_tan + 0.3*psi_los);
    elseif path_idx == 2
        carrot = chasing_line([x_s_t, y_s_t], [x_f_t, y_f_t], car_x, car_y, delta_lahead, i);
```

```
        psi_d = atan2(carrot(2)-car_y, carrot(1)-car_x);
    else
        [carrot, th_carrot, sgn] = chasing_arc(x_f_c, y_f_c, the_0_f_t, the_0_f_f, C2, r_radius, car_x, car_y, lamuda_lahead);
        psi_tan = wrapToPi(th_carrot + sgn*pi/2);
        psi_los = atan2(carrot(2)-car_y, carrot(1)-car_x);
        psi_d = wrapToPi(0.7*psi_tan + 0.3*psi_los);
    end

    % heading control
    epsi = wrapToPi(psi_d - car_psi);
    % Calculate angular velocity (u) - it is not control law , it is a rate control by gain
    % formula: κ=ω/v -> κ=kψeψ = ω=kψeψv;
    u = kpsi * epsi * v;
    u = max(min(u, ang_v_max), -ang_v_max);

    % dynamics
    car_x = car_x + v * cos(car_psi) * dt;
    car_y = car_y + v * sin(car_psi) * dt;
    % car_psi = car_psi + (u/v) * dt;
    car_psi = car_psi + u * dt;

    % draw the animation of car movement
    set(car_p, 'XData', car_x, 'YData', car_y);
    set(arrow, 'XData', car_x, 'YData', car_y, 'UData', 2*cos(car_psi), 'VData', 2*sin(car_psi));
    set(traj, 'XData', x_hist(1:i), 'YData', y_hist(1:i));
    drawnow;

    % Main Path State Check: the car is close to final point, state change to next part
    if norm([car_x - x_f, car_y - y_f]) < eps
        if seg < size(D_list, 1)
```

```
            % Next State Update
            seg = seg + 1;
            path_idx = 1;
            % --------  Update Path Segment Parameters  --------
            [idx, ...
            x_s, y_s, th_s, ...
            x_f, y_f, th_f, ...
            x_s_c, y_s_c, ...
            x_f_c, y_f_c, ...
            d, ...
            x_s_t, y_s_t, the_0_s_t, ...
            x_f_t, y_f_t, the_0_f_t, ...
            the_0_s_s, the_0_f_f, ...
            C1, C2] = data.parse_D_list(D_list(seg,:));
            % draw the straight line as path
            plot([x_s_t x_f_t],[y_s_t y_f_t], 'y--', 'LineWidth', 1.5);
          else
            break;
          end
        end

    end

end

%% ================================================================
%  Get Carrot in ARC
% ================================================================
function [carrot, th_carrot, sgn] = chasing_arc(cx , cy, th_start, th_end, C, r, car_x, car_y,
look_ahead)
```

```
% Arc length calculation, sgn showing the turning direction, detal showing the turning length
if char(C) == 'L'
   arc_length = wrapTo2Pi(th_end - th_start);
   sgn = +1;
else
   arc_length = wrapTo2Pi(th_start - th_end);
   sgn = -1;
end

% Projection car position to circle
th_car = atan2(car_y - cy, car_x - cx);

% Car progress evaluation: Calculate the distance between start and final
if char(C) == 'L'
   car_progress = wrapTo2Pi(th_car - th_start);
else
   car_progress = wrapTo2Pi(th_start - th_car);
end

% Just in case to prevent progress greater than total distance of arc
car_progress = min(max(car_progress, 0), arc_length);

% carrot length in arc - Δθ=s/r
% dth_la = look_ahead / r;
dth_la = look_ahead;

% carrot theta respect to progress
prog2 = min(car_progress + dth_la, arc_length);

% carrot theta respect to zero
th_carrot = th_start + sgn * prog2;
```

```
    % carrot position
    carrot = [cx + r*cos(th_carrot), cy + r*sin(th_carrot)];



    %plot(carrot(1), carrot(2), 'o', 'Color', [1 0.5 0], "MarkerSize",5, "LineWidth",5);

end

%% ================================================================
%  Get Carrot in Line
% ================================================================
function carrot = chasing_line(t_entry, t_exit, car_x, car_y, look_ahead, i)

    % distance of tangent line
    dtx = t_exit(1) - t_entry(1);
    dty = t_exit(2) - t_entry(2);
    d_t = norm([dtx, dty]);

    % direction of tangent line （unit tangent vector）
    utv = [dtx, dty] / d_t;

    % progress of the car in the tangent path - calculate the projection of car position
    car_progress = dot([car_x, car_y] - t_entry, utv);
    car_progress = min(max(car_progress, 0), d_t);

    % carrot position
    carrot = t_entry + min(car_progress + look_ahead, d_t) * utv;

    if mod(i, 200) == 0
        plot(carrot(1), carrot(2), 'o', 'Color', [1 0.5 0], "MarkerSize",4, "LineWidth",5);
```

```
    end
    % plot(carrot(1), carrot(2), 'ro', "MarkerSize",5, "LineWidth",5);

end
```

### +common / init_map.m

```
function init_map(env, no)

figure(no); hold on; grid on;
axis([env.x_min env.x_max env.y_min env.y_max]);

% Draw the start and end point
common.start_end_plot(env);

% Draw obstacles
common.obstacles_poly_plot(env.poly);

end
```

### +common / make_car.m

```
function car = make_car(start)

% Car initial position
car.x = start(:,1);    %[m]
car.y = start(:,2);    %[m]

% Car initial heading
```

```
car.th = deg2rad(45);   %[rad]

% Car Initial State
car.state = [car.x, car.y, car.th];

% Car minimum return radius
car.r = 5;          %[m]

% Linear velocity
car.v = 5;              %[m/s]

% Maximum angular velocity
car.w_max = 1.0;        %[rad/s]

% Maximum lateral acceleration
car.u_max = 5.0;        %[m/s^2]

end
```

+common / make_env.m

```
function env = make_env()
   % Figure max size
   env.x_min = -5;
   env.y_min = -5;
   env.x_max = 1000;
   env.y_max = 1000;

   % Obstacle position
   env.poly = {[300, 500; 200, 200; 450, 300], [550,450; 530,300; 700,250; 750, 350], ...
```

```
        [600,700; 650,500; 750,600; 800,780], [310,750; 340,600; 500,550; 480,660]};

    % Initial position
    env.start = [0, 0];

    % Goal position
    env.goal = [600, 600];
    % env.goal = [200, 200];

    % Goal reaching tollerance
    env.goal_radius = 50.0;

end
```

+common / obstacles_poly_plot.m

```
%% ================================================================
%  Visualize Obstacles
% ================================================================
function obstacles_poly_plot(poly)
    shape_count = size(poly, 2);
    for i = 1:shape_count
        crnt_poly = poly{i};
        poly_size = size(crnt_poly);  % the ith perticular segments

        % check dimension is 2
        if poly_size(2) ~= 2
            return;             % error, incorrect polymer definition
        end

        % check line segement return to start
        if any(crnt_poly(poly_size(1), :) ~= crnt_poly(1, :))
```

```
        poly_size(1) = poly_size(1)+1;
        crnt_poly(poly_size(1), :) = crnt_poly(1, :);
      end
      %seg_count = poly_size(1) - 1;

      plot(crnt_poly(:,1), crnt_poly(:,2),'b--','LineWidth',2); hold on;
    end
end
```

### +common / start_end_plot.m

```
function start_end_plot(env)

% Draw the start point
plot(env.start(1), env.start(2), 'ro',"LineWidth", 2, "MarkerSize", 8, "MarkerFaceColor", 'r');

% Draw the goal point
plot(env.goal(1), env.goal(2), 'ko',"LineWidth", 2, "MarkerSize", 8, "MarkerFaceColor", 'k');

end
```

### +data / parse_D_list.m

```
function [idx, ...
 x_s, y_s, th_s, ...
 x_f, y_f, th_f, ...
 x_s_c, y_s_c, ...
 x_f_c, y_f_c, ...
 d, ...
 x_s_t, y_s_t, the_0_s_t, ...
 x_f_t, y_f_t, the_0_f_t, ...
 the_0_s_s, the_0_f_f, ...
```

```
 C1, C2] = parse_D_list(D)
% D (1 x 22) layout
% -------------------------------------------------------------------------
%  1  : idx            (path type index, e.g., LSL/RSR/...)
%
%  2  : x_s            (start waypoint x)
%  3  : y_s            (start waypoint y)
%  4  : th_s           (start heading angle)
%
%  5  : x_f            (final waypoint x)
%  6  : y_f            (final waypoint y)
%  7  : th_f           (final heading angle)
%
%  8  : x_s_c          (start circle center x)
%  9  : y_s_c          (start circle center y)
%
% 10  : x_f_c          (final circle center x)
% 11  : y_f_c          (final circle center y)
%
% 12  : d              (total Dubins path distance)
%
% 13  : x_s_t          (start tangent point x)
% 14  : y_s_t          (start tangent point y)
% 15  : the_0_s_t      (start circle → tangent exit angle)
%
% 16  : x_f_t          (final tangent point x)
% 17  : y_f_t          (final tangent point y)
% 18  : the_0_f_t      (final circle → tangent entry angle)
%
% 19  : the_0_s_s      (start circle → start pose angle)
% 20  : the_0_f_f      (final circle → final pose angle)
```

```
%
% 21  : C1              (start turn direction: 'L' or 'R')
% 22  : C2              (final turn direction: 'L' or 'R')
% ------------------------------------------------------------------------


   idx =  D(1);
   x_s = D(2);
   y_s = D(3);
   th_s = D(4);
   x_f = D(5);
   y_f = D(6);
   th_f = D(7);
   x_s_c = D(8);
   y_s_c = D(9);
   x_f_c = D(10);
   y_f_c = D(11);
   d = D(12);
   x_s_t = D(13);
   y_s_t = D(14);
   the_0_s_t = D(15);
   x_f_t = D(16);
   y_f_t = D(17);
   the_0_f_t  = D(18);
   the_0_s_s  = D(19);
   the_0_f_f = D(20);
   C1 = D(21);
   C2 = D(22);


end
```